ASP.NET Core 8

Ali Ilci

B. A.

Einleitung

- Kursablauf & Organisatorisches
- Räumlichkeiten
- Vorstellungsrunde
  - Hintergrund, Motivation und Vorkenntnisse
- Bei Unklarheiten: Möglichst sofort fragen!

Über mich – Ali Ilci

- Freiberuflicher IT - Trainer (MCT)
- Fachinformatiker
- Erziehungswissenschaftler
- Themenschwerpunkte:
  - ASP.NET Core / MVC / WebApi / Blazor
  - C#
  - Html5 & Css3
  - JavaScript / TypeScript

- www.ilci.de

.NET Core

Was ist .NET Core

- Plattformübergreifendes Framework mit dem Ziel so universell wie möglich eingesetzt zu werden
- Open Source
- Bibliotheken werden durch NuGet verteilt
- Typische Einsatzszenarien sind zurzeit ASP.NET Webanwendungen, Konsolen-Apps sowie UWP-Apps
- Ist keine Untermenge des .NET Framework
- Besitzt ein Command Line Interface (CLI)

Full .NET Framework vs. .NET Core

Full .NET Framework	.NET Core
Vollständiges „etablierte“ Framework	Modulare Version des .NET Framework
Nur Windows	Cross-Plattform
	Keine Untermenge vom .NET Framework
	Windows, Linux, Mac
	Implementation des .NET Standard

Entscheidung zwischen Full .NET Framework vs. .NET Core

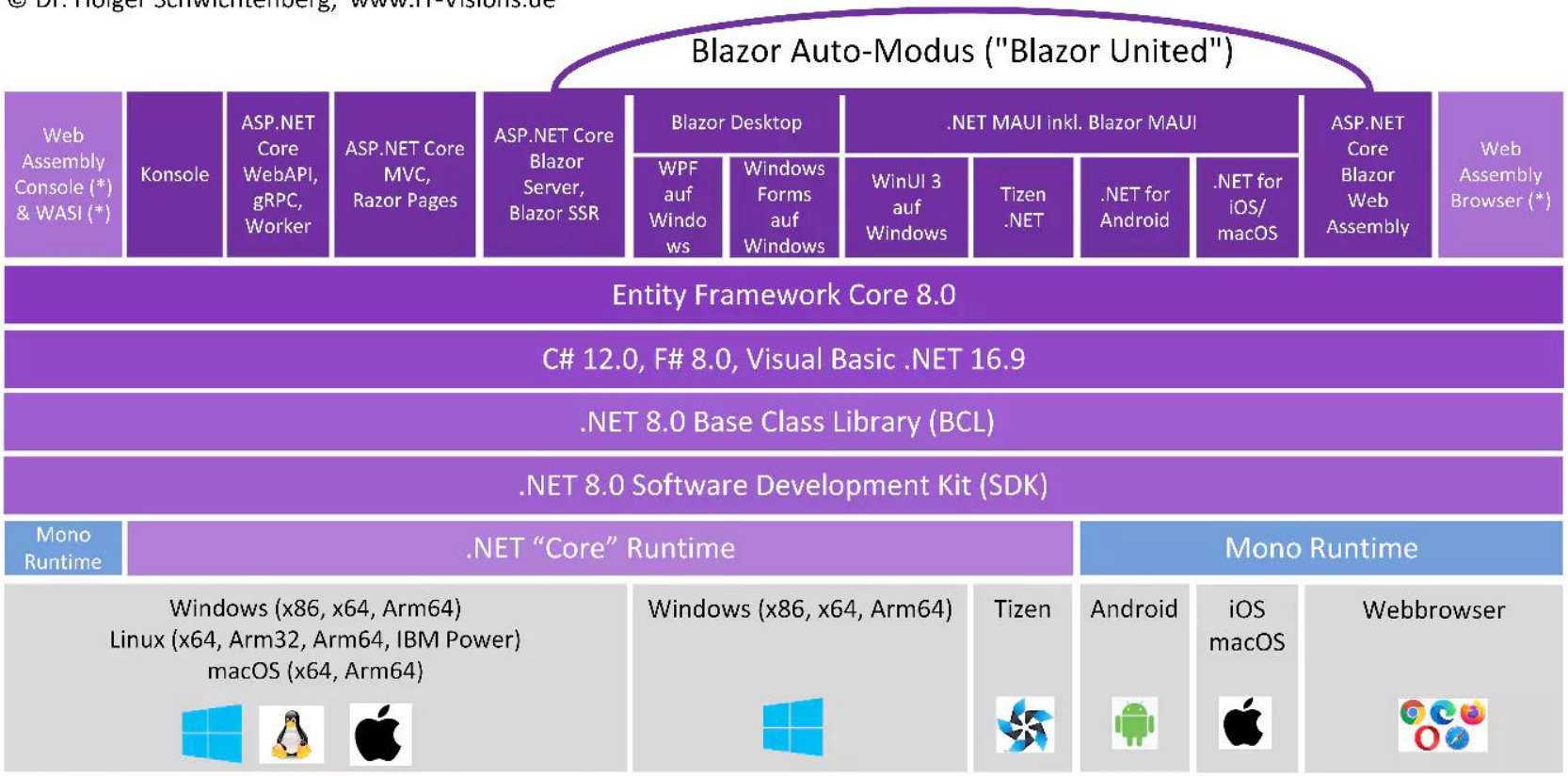
Full .NET Framework	.NET Core
Aktuelle Projekt(e) nutzen .NET Framework	Cross-Plattform
Third-Party .NET Libraries	Docker Container
.NET Technologien, die nicht für .NET Core vorhanden sind	Side-by-Side .NET Versions in Applikationen

https://docs.microsoft.com/de-de/dotnet/articles/standard/choosing-core-framework-server

.NET Familie 2023/2024

Aufbau von .NET 8.0

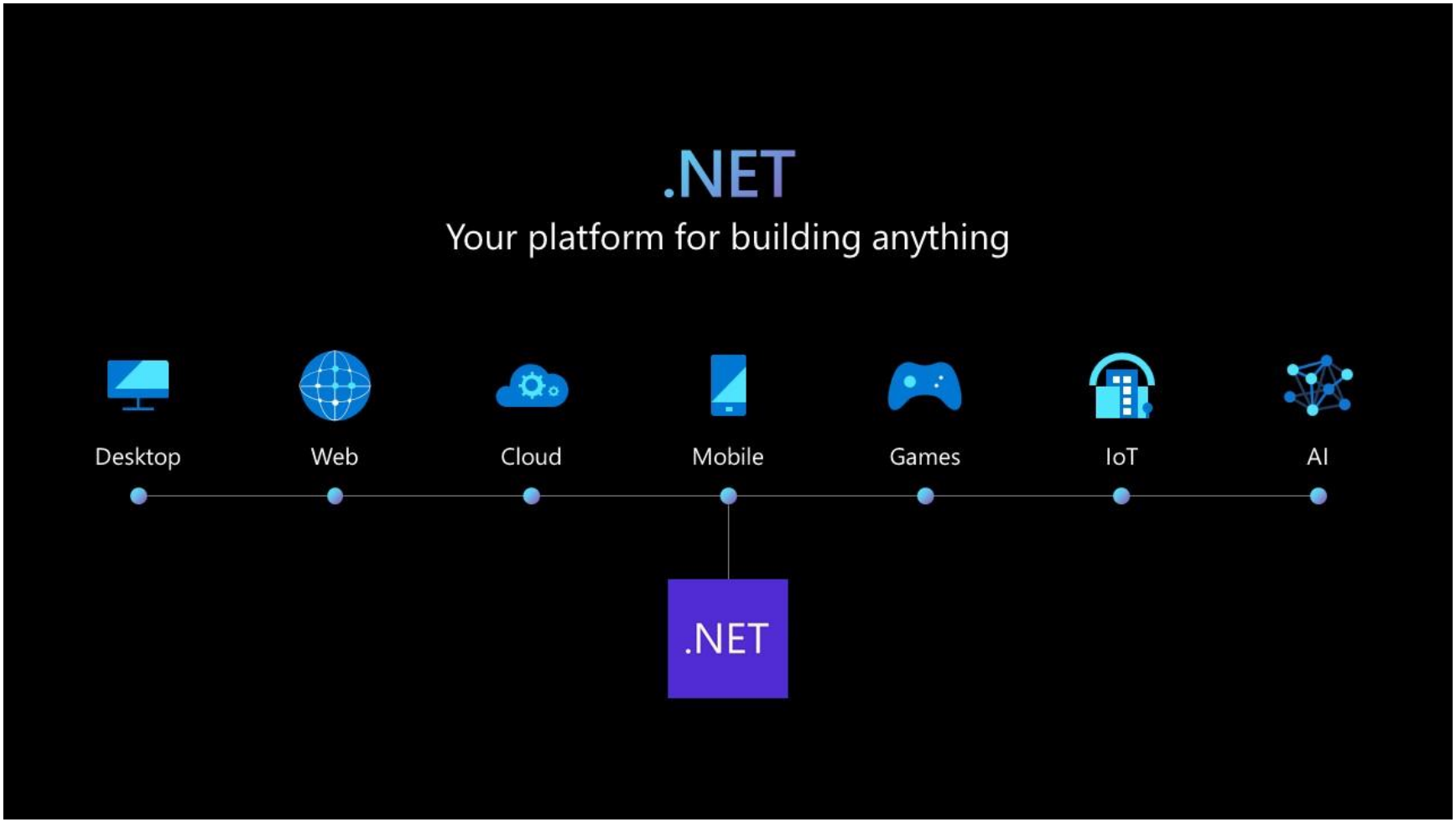
© Dr. Holger Schwichtenberg, www.IT-Visions.de



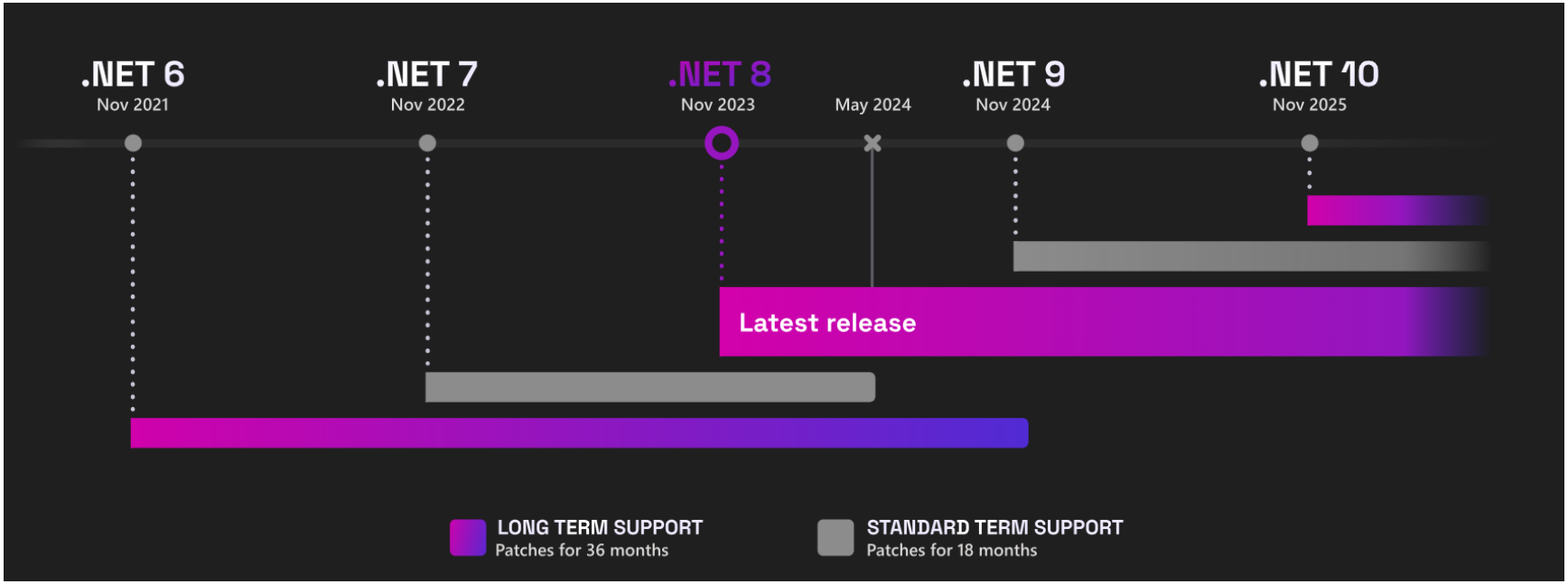
(*) Experimentelle Implementierungen ohne Support seit .NET 7.0 und auch noch in .NET 8.0

https://www.heise.de/blog/Neu-in-NET-8-0-2-Neue-Anwendungsarten-9581213.html
Holger Schwichtenberg Stand: 22.Dezember 2023

.NET 8 Platform



.NET Schedule



https://dotnet.microsoft.com/en-us/platform/support/policy

.NET CLI

- Command Line Application, die zum Entwickeln von Anwendungen auf allen Plattformen verwendet wird
- Führt die App aus und hosted die CLR
- OpenSource / SDK
- Erweiterbar (z.B. Entity Framework Befehle)
- Dokumentation
  - https://docs.microsoft.com/de-de/dotnet/core/tools/

.NET CLI Befehle „dotnet“ (Auszug)

Befehl	Beschreibung
new	Erstellt ein neues Projekt, eine Konfigurationsdatei oder eine Lösung auf Grundlage der angegebenen Vorlage.
restore	Stellt die Abhängigkeiten und Tools eines Projekts wieder her
build	Erstellt ein Projekt und alle seine Abhängigkeiten
publish	Packt die Anwendung und ihre Abhängigkeiten in einen Ordner für die Bereitstellung auf einem Hostsystem
run	Führt Quellcode ohne explizite Kompilierungs- oder Startbefehle aus.
test	.NET-Testtreiber, der verwendet wird, um Komponententests auszuführen.
pack	Packt den Code in ein NuGet-Paket
clean	Löscht die Ausgabe eines Projekts.
sln	Ändert eine .NET Core-Projektmappendatei.
store	Speichert die angegebenen Assemblys im Laufzeitpaketspeicher

Themen

Repository

Tools

ViewComponents

Bootstrap

AutoMapper

Model

Web Technologien

Filter

JQuery

Views

TagHelper

Logging

Mobile Views

Authentifizierung

Middleware

Routing

AJAX

Partial Views

Validation

Jquery UI

Fehlerbehandlung

Actions

Testing

Razor

HTML Helpers

Action Filter

Entity Framework Core

Dependency Injection

Asynchronität

Security

Controller

Data Annotations

Best Practices

Responsive Web Design

C# 10/11

C# 10 / 11 / 12

Exkurs

Implicit & Global Using Statements

- Vermeidet wiederholte Auflistung von Usings
- Aktivierbar in Projektdatei (Default-Einstellung)
  `<ImplicitUsings>enable</ImplicitUsings>`
- Globale Usings um Namespace projektweit bekannt zu machen

```
//Vorher
using System;
using System.Collecions.Generic;
using Microsoft.AspNetCore.Http

//Neu mit Implicit Using Statements
-

//Neu mit Global Using Statements in separater CS-Datei
global using Microsoft.AspNetCore.Http
```

File Scoped Namespaces

- Einrückung der Namespaces
- Konfigurierbar für gesamte Solution mit .editorconfig

```
//.editorconfig
csharp_style_namespace_declarations = file_scoped
dotnet_diagnostic.IDE0161.severity = suggestion
```

```
namespace MeinNamespace.Services
{
    public class MeineKlasse
    {}
}

//File Scoped Namespaces
namespace MeinNamespace.Services;

public class MeineKlasse
{}
```

MaxBy/MinBy & DateOnly and TimeOnly

- **DateOnly** bzw. **TimeOnly** repräsentieren entweder das Datum **oder** die Uhrzeit im Gegensatz zu DateTime
- **MinBy/MaxBy** geben ganzes Objekt zurück

```
//DateOnly & TimeOnly
DateOnly date = DateOnly.MinValue; //01.01.0001 ohne Zeit
TimeOnly time = TimeOnly.MinValue; // 12:00

//MaxBy/MinBy
List<Person> people = new List<Person>
{
  new Person { Name = "John Doe", Alter = 25},
  new Person { Name = "Jane Doe", Alter = 23}
}

var person = people.MaxBy(c => c.Alter); // John Doe
```

Raw String Literal / Required Member

- Unformatierte Zeichenfolgen-Literale können beliebigen Text enthalten, ohne dass Escapezeichen erforderlich sind
- Required – Modifizierer gibt an, dass das Feld / Eigenschaft von allen Konstruktoren oder via Objektinitializer initialisiert werden muss

```
//String Literal vor C#11
var message = "Der folgende Text ist sehr \"wichtig\" ".

//ab C#11
var message = """Der folgende Text ist sehr "wichtig" """.


//Required Member
public required string FirstName {get; init;}
```

Primärkonstruktoren C# 12

- Verkürzte Schreibweise von Konstruktoren. Achtung parameterloser Konstruktor ist nicht mehr vorhanden und muss separat deklariert werden. Dadurch Aufruf des Primärkonstruktors

```
public class Kunde(Guid kundeId, string name, float preis)
{
   public Guid KundeId { get; set; } = kundeId;
   public string Name { get; set; } = name;
   public Kunde() : this(Guid.Empty, "") { }

   public override string ToString()
   {
    return $": {Name} {preis}";
   }
}
```

Vereinfachte Initialisierung von Mengen, Spread, Opt. Parameter in Lambdas C# 12

- Syntax mit eckigen Klammern wie in Javascript
- Spread-Operator möglich (Array aus anderen Arrays)
- Optionale Parameter in Lambdas

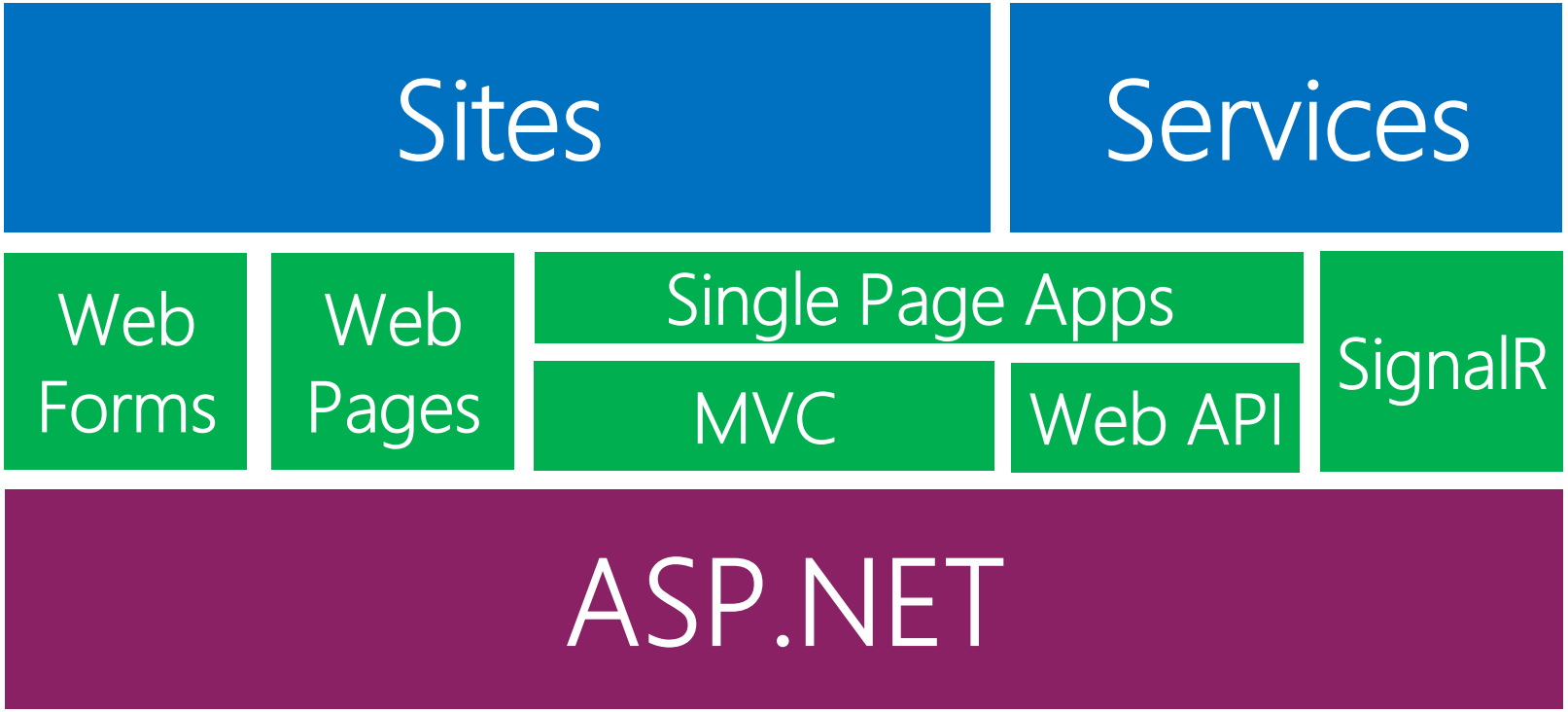
```
// Vorher
string[] fruechte = new string[] {"Banane", "Orange"};

//Nachher
string[] fruechte = ["Banane", "Orange"];

//Spread-Operator
string[] alleFruechte = [.. fruechte, "Apfel"];

//Lambda mit opt. Parameter
var bru = (decimal x, decimal mw = 1.19) => (x * mw);
```

ASP.NET Core 8

Microsoft Web Technologien

Entwicklertools	Host	Code Ausführtechnologien	
Visual Studio	IIS	**Server-Side**	**Client-Side**
WebMatrix	SQL Server	ASP.NET	JavaScript
Visual Studio Code	Windows Azure	ASP.NET Core	JQuery
		ASP.NET Blazor Server	AJAX
			Knockout
			TypeScript
			ASP.NET Blazor Webassembly

Überblick ASP.NET 4.5



Überblick ASP.NET Core 8

ASP.NET Core 8
(MVC + WebAPI + MinimalApi + Razor Pages + SignalR + gRPC + Blazor (SSR, Server, Blazor WebAssembly)

C# 12

.NET 8 Base Class library

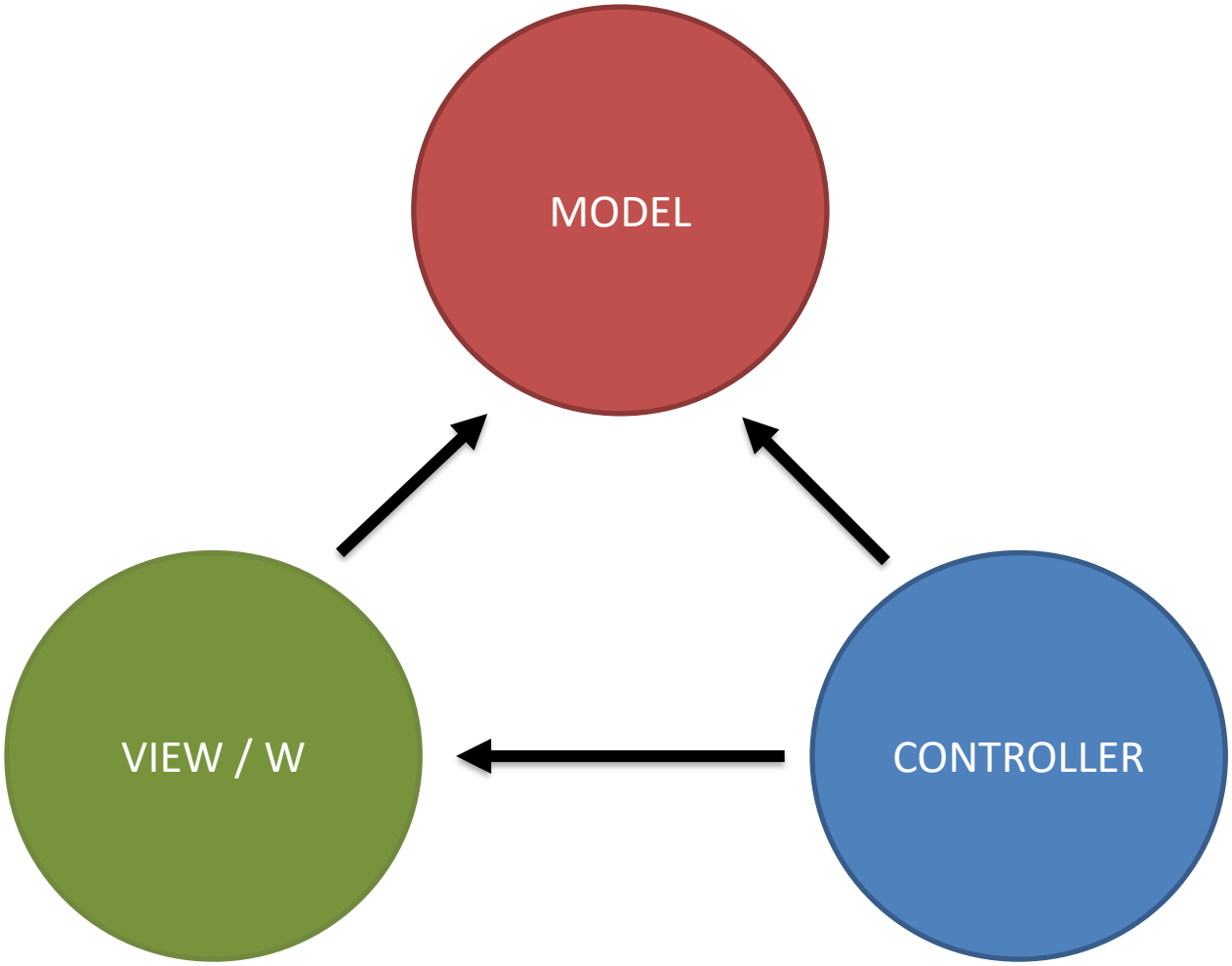
.NET Core
Windows & Linux & MacOS

ASP.NET Core MVC

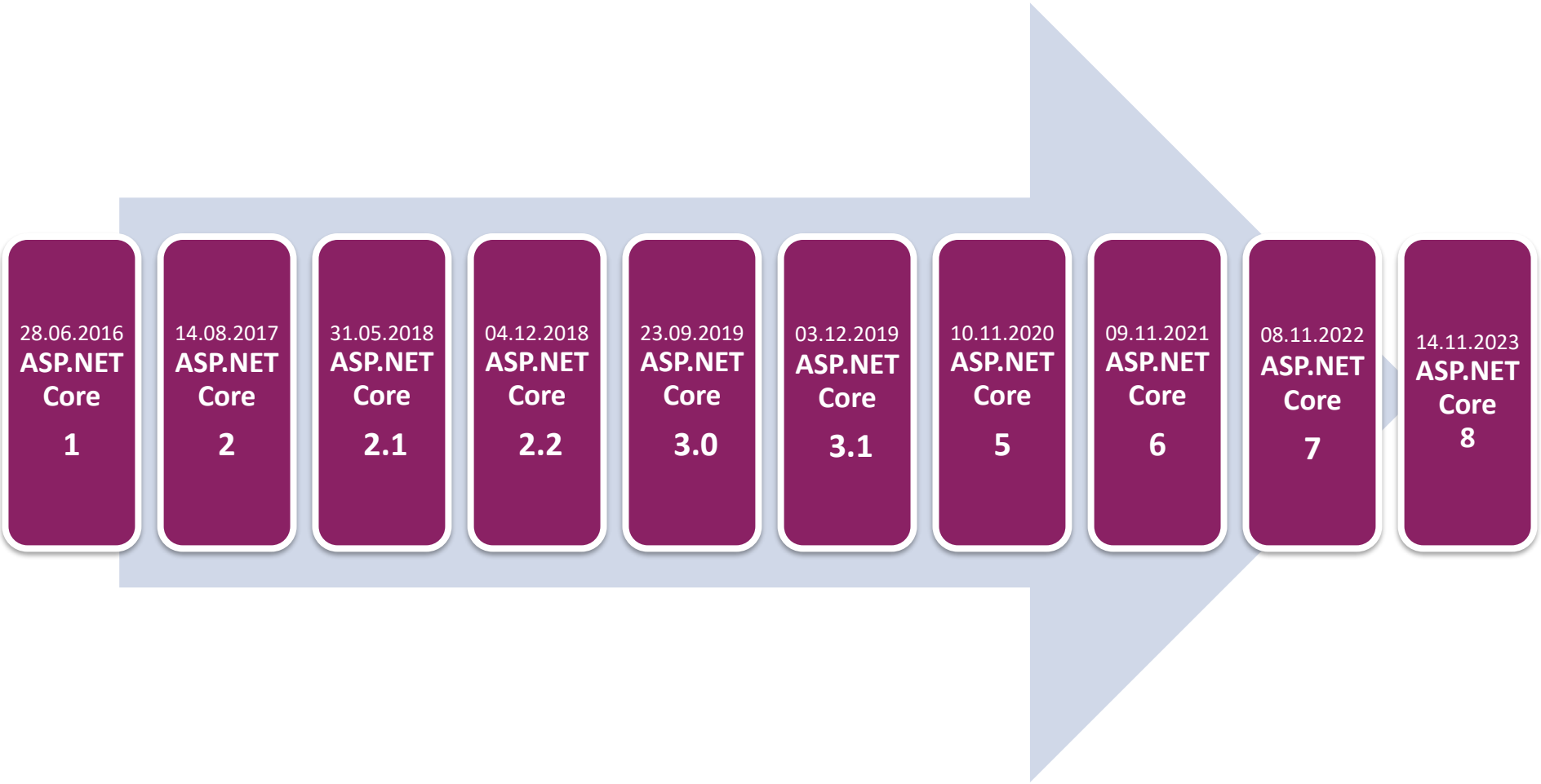
Was ist MVC?

- **M**odel-**V**iew-**C**ontroller
- Entwurfs-/Architektur zur Strukturierung von Software-Projekten
- Erstmals 1979 in Smalltalk
- Ziel ist die Separation of Concerns
  - Datenmodell (Model)
  - Präsentation (View)
  - Steuerung (Controller)

Model-View-Controller



Versionshistorie ASP.NET Core MVC



Unterschiede ASP.NET Web Forms & MVC

Web Forms	MVC
Control und ereignisbasiert	Separation Of Concerns
Rich UI-Controls	Keine Toolbox
Adaptives Rendering	Volle Kontrolle über HTML / Urls
.aspx Dateien und Code-behind	.cshtml und .cs Dateien
RAD Development	Unit Testing / TDD
ViewState / Postbacks	Kein ViewState / Postback

Hands On

Projekt „Teilnehmer“

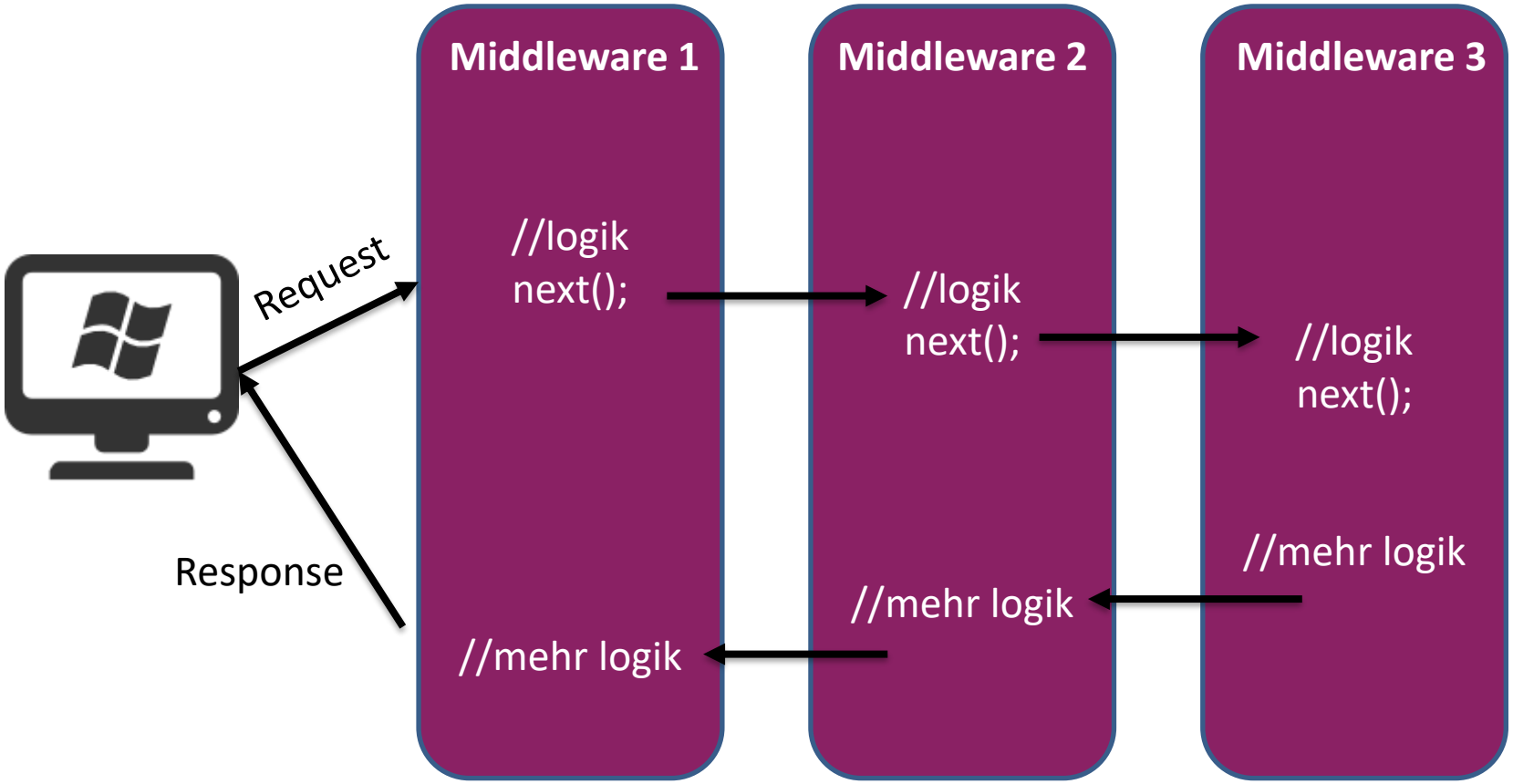
Middleware

ASP.NET Core

- **Program**-Klasse
  - Verantwortlich für die Konfigurierung/Ausführen der Anwendung
  - Einstiegspunkt der Webapplikation
- **WebApplicationBuilder** verantwortlich für Configuration und Service Registration
- **IServiceCollection** wird genutzt um Services dem Container hinzuzufügen und zu konfigurieren
- Spezifizierung wie eine ASP.NET Applikation auf individuelle HTTP-Anfragen zu antworten
- Middlewares um HTTP-Anfrage-Pipeline zu konfigurieren

ASP.NET Core Request Pipeline & Middleware

- Querschnittsfunktion ähnlich HttpModulen & MessageHandler
- Erste Anlaufstelle bei der Bearbeitung der Anfragen.



Middleware

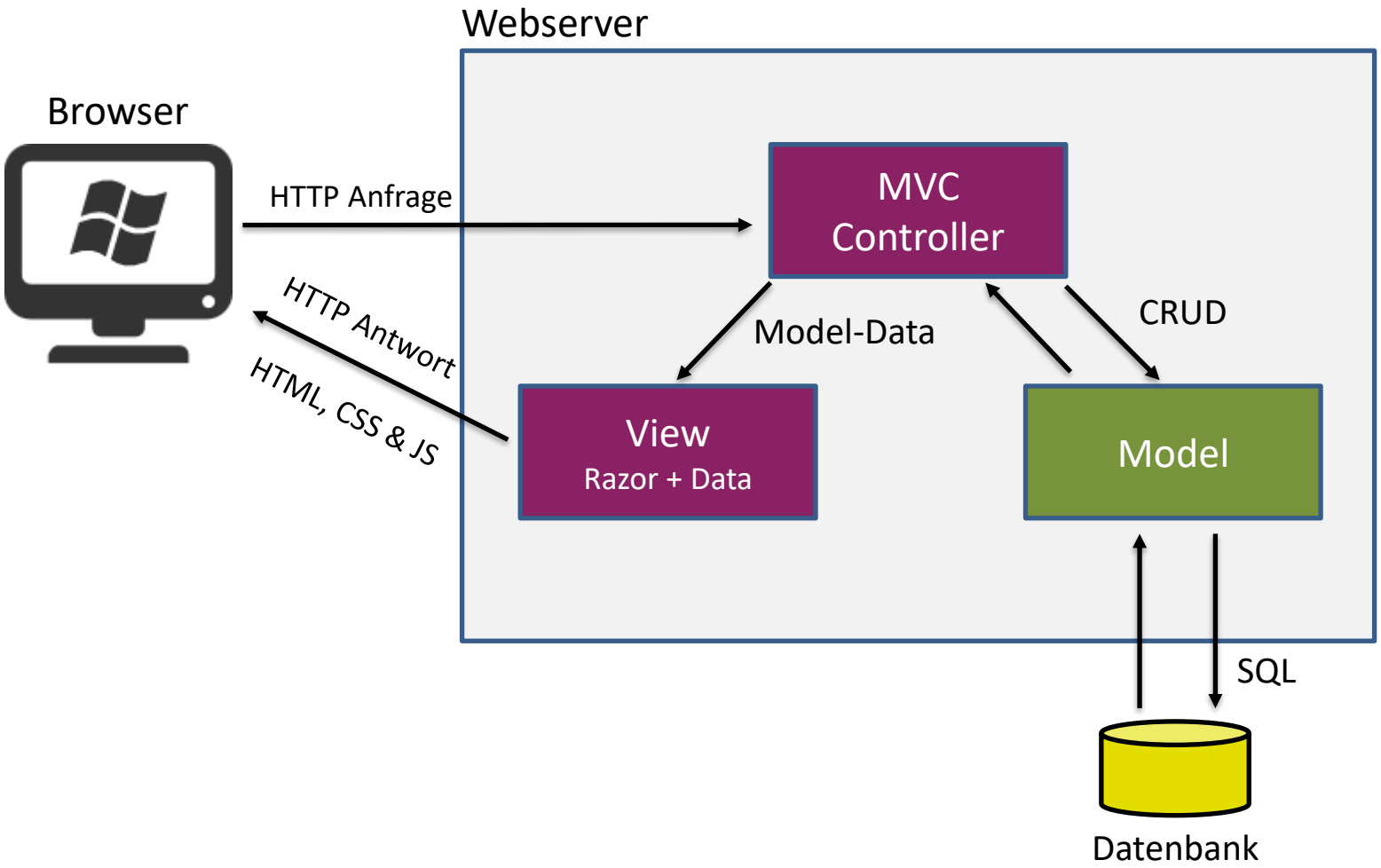
- Software, die zu einer Anwendungspipeline zusammengesetzt wird (Delegate)
- Anwendung durch Extension-Methoden des **WebApplication** Objekts
- Die **WebApplication** stellt den Mechanismus bereit die Request-Pipeline zu konfigurieren
- Built-In-Middleware
  - Mvc, StaticFiles, CORS, Routing, DeveloperExceptionPage etc.
- Custom Middleware möglich
- Reihenfolge des Aufrufs relevant

Hands On

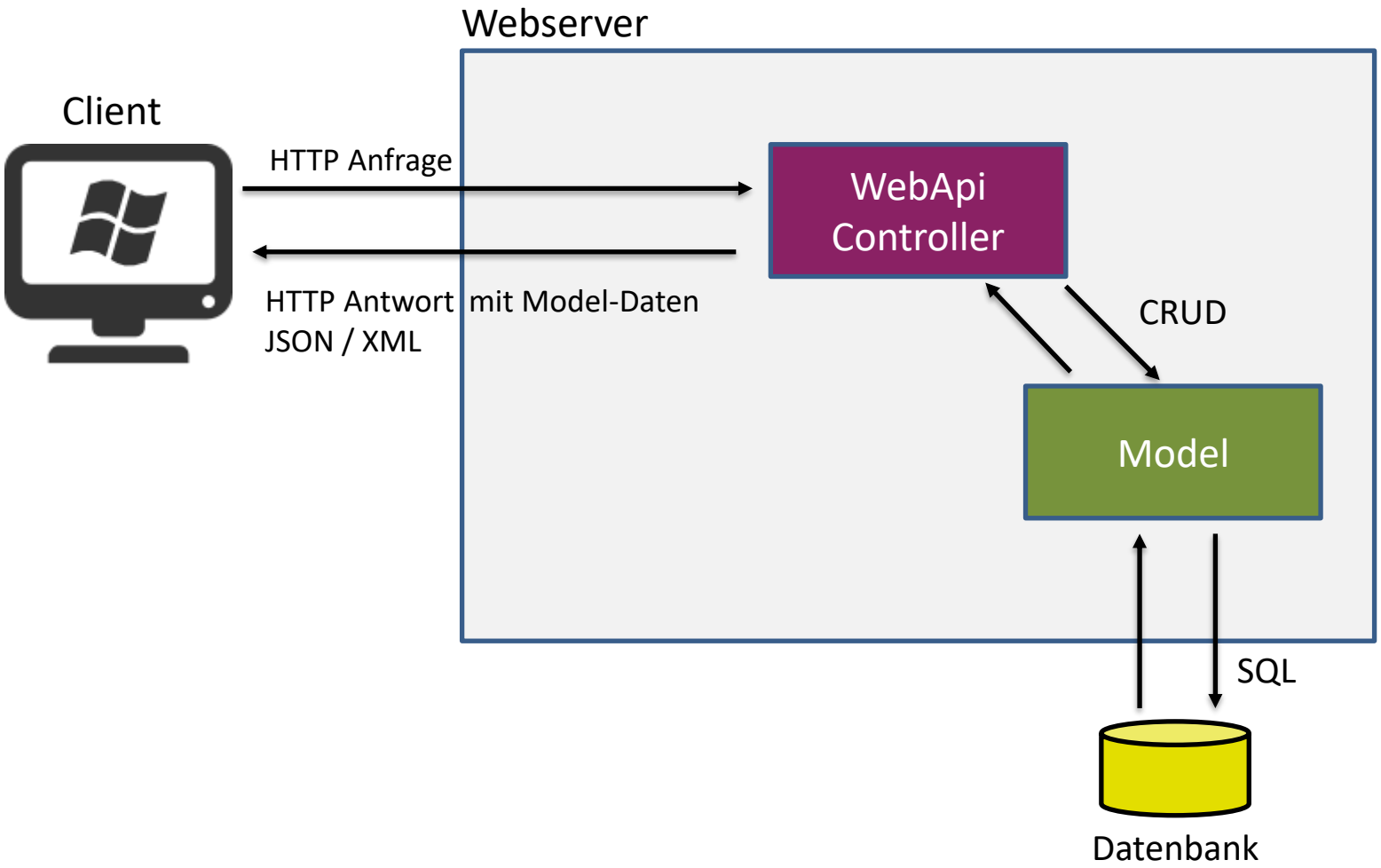
Projekt „Teilnehmer“ Middleware

Controller & Actions

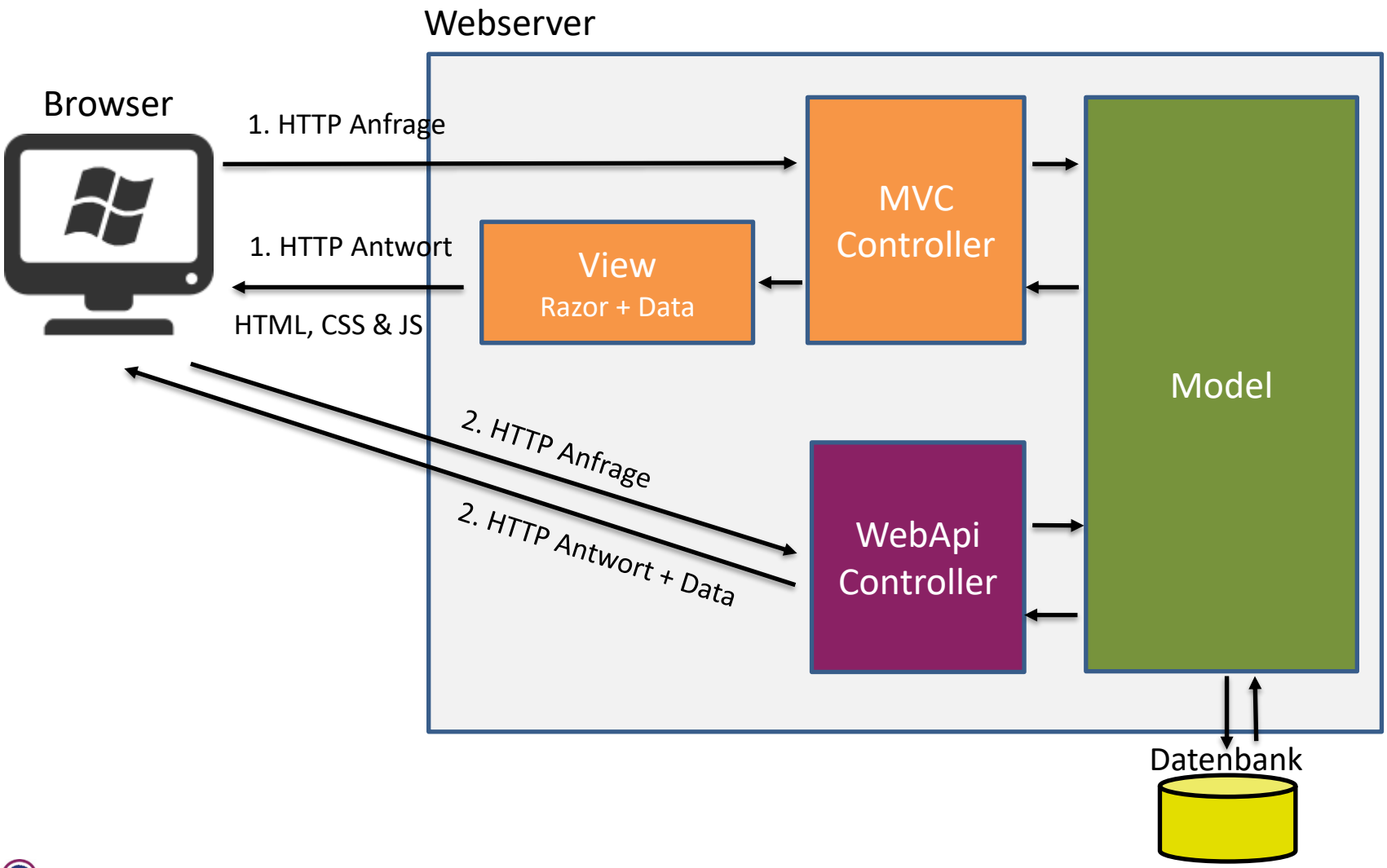
Request-Response Ablauf ASP.NET Core MVC



Request-Response-Ablauf ASP.NET Core WebApi

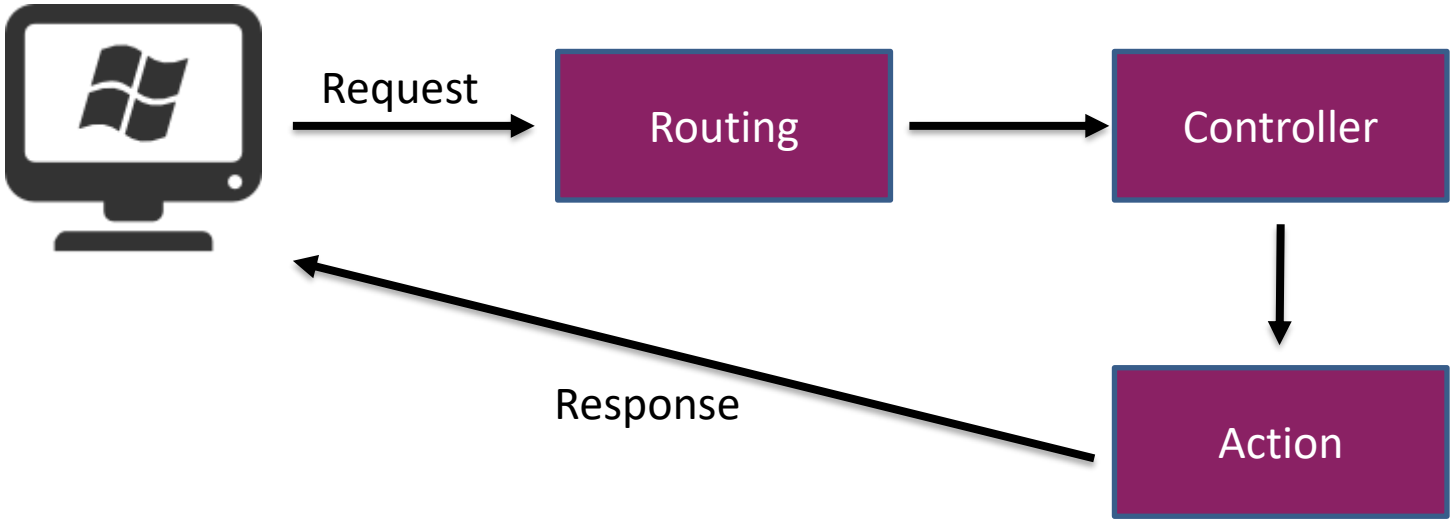


Request-Response Single Page Application



Was ist die Aufgabe eines Controller?

- Ausführung einer Aktionsmethode (meist CRUD)



Action Results

- Ein **ActionResult** ist das Ergebnis einer **ActionMethode** welches die HTTP-Antwort zum Client darstellt
- Implementiert die Schnittstelle **IActionResult**
- Folgende Rückgabetypen können zurückgegeben werden
  - void (Keine Rückgabe)
  - IActionResult (und abgeleitete ActionResults)
  - anderen Typ

Action-Result Typen

Rückgabetyp	Beschreibung
Content	Liefert den angegebenen String als Ergebnis der Anfrage zurück
File	Veranlasst einen Download als Ergebnis
JavaScript	Gibt den angegebenen String als JavaScriptCode zurück, der auf dem Client ausgeführt wird
Json	Liefert das angegebene Modell JSON-formatiert zurück
PartialView	Verwendet eine partielle View, um einen Teil einer Seite zu rendern
RedirectToAction	Leitet die aktuelle Anfrage an die angegebene Action-Methode um
Empty	Tut nichts!
RedirectToRoute	Leitet die aktuelle Anfrage zu dem URL um, der zur angegebenen Route passt
View	Ruft eine View auf

Actions mit Parametern

- Der **DefaultModelBinder** ermittelt den Namen des Parameters und ordnet ihn der Action-Methode zu wegen Namensgleichheit
- **http://www.webseite.de/kunde/getkunde?name=schmidt**

DefaultModelBinder

```
public IActionResult GetKunde(string name)
{
    var kunde = (from k in db.Kunden
                         where k.Name == name
                         select p).FirstOrDefault();

   return View("Details",kunde);
}
```

Daten an Views

- 2 Möglichkeiten um Daten an View zu übergeben
  - **View() Helper Methoden** bei Modeldaten
  - **ViewBag/ViewData** Eigenschaft im Falle von fehlenden Modeldaten

Controller

```
ViewBag.Uhrzeit = DateTime.Now;
ViewBag.Message = "Dies ist ein schöner Tag";
```

View

```
<p>
     Die Nachricht des Tages lautet: @ViewBag.Message
     Heute ist der @ViewBag.Uhrzeit
</p>
```

Eigene Action Results

- Eigene Klassen müssen von **`IActionResult`** oder **`ActionResult`** erben und **`ExecuteResultAsync`** bzw. **`ExecuteResult`**`(synchron)` implementieren
- Alternative erben von vorhandenen ActionResults

```
public class NoContentResult : IActionResult
{
  public Task ExecuteResultAsync(ActionContext context)
  {
    return Task.FromResult(context.HttpContext.Response.StatusCode =
                                StatusCodes.Status204NoContent);
  }
}
```

Asynchrone Controller

- Vereinfachte Implementierung ab .NET 4.5 mit async/await

```
public async Task<IActionResult> GetAutor(int id)
{
    var autor = await db.Autoren.FindAsync(id);

    if(autor == null)
    {
       return NotFound();
    }

    return View(autor);
}
```

Views

Razor, Layout, Html Helper, Bootstrap, TagHelper, ViewComponents

Was ist eine View?

- Ansichten / Views sind in ASP.NET Core MVC Vorlagen in HTML und einer speziellen CodeSyntax bzw. View-Engine namens RAZOR @
- Zwecks Übersichtlichkeit und Wiederverwendbarkeit kann ein Template weitere untergeordnete Templates beinhalten
- Dateien der Views haben die Endung cshtml
- Ansichten haben eine **`Model`** Eigenschaft, dessen Typ durch @model deklariert wird

Razor-Syntax

Syntax	Beschreibung

Code Block	`@ {`
	`  int x = 666;`
	`  string y = "Dies ist ein Text";`
	`}`

Ausdruck (HTML Codiert)	`<span>@model.Message</span>`

Ausdruck (uncodiert)	`<span>`
	
	`@Html.Raw(model.Message)`
	`  </span>`

Kombinieren Text und markup	`@foreach(var item in items)`
	`{`
	`  <span>@item.Eigenschaft</span>`
	`}`

Code and Klartext mischen	`@if (foo)`
	`{`
	`  <text>Klartext</text>`
	`}`

Razor-Syntax II

Syntax	Beschreibung

Using block	`@using (Html.BeginForm())`
	`{`
	`  <input type="text" value="input hier">`
	`}`

Code and Klartext mischen (alternativ)	`@if (foo)`
	`{`
	`  @:Plain Text is @bar`
	`}`

Escape @	`Hi philha@@example.com`
Kommentar	`@* This is a server side multiline comment *@`
Generische Methode aufrufen	`@(MyClass.MyMethod<AType>())`

URL Auflösung mit ~	`<img src="~/images/myImage.jpg"`
	`wird ersetzt zu`
	`<img src="/meineApp/images/myImage.jpg"`

Hands On

Razor Syntax

ASP.NET MVC Models

Überblick

- Was ist ein Model?
- Erstellung von Models
- Data Annotations
- Validation
- Entity Framework

Was ist ein Model?

Eine Klasse

Wer nutzt das Model?

- Benutzer
  - Mit welchen Daten wird interagiert?

- ASP.NET MVC
  - Eingabesteuerelemente
  - Darstellung
  - Validation

Model

Author
- Id : Guid
- Name : string
- Description : string
- BirthDate: DateTime

```
public class Author
{
 public Guid Id { get; set; }
 public string Name { get; set; }
 public string Description {get;set;}
 public DateTime? BirthDate {get;set;}
}
```

Data Annotations

- Steuern die Anzeige und die Validation von Modeleigenschaften (Properties)
- System.ComponentModel.DataAnnotations
- Built-In-Data-Annotations für Anzeige

Anzeige Attribute	Beschreibung
DataType	Legt den Datentyp fest
Display	Legt den Anzeigenamen fest
DisplayFormat	Legt fest wie Datenfelder dargestellt und formatiert werden
UIHint	Bestimmt das Control für die Modeleigenschaft

Data Annotations für Anzeige

Data Type Enumeration	Beschreibung
CreditCard	Repräsentiert eine Kreditkartennummer
Currency	Repräsentiert eine Währung
Date	Repräsentiert ein Datum
DateTime	Repräsentiert ein Datum mit Uhrzeit
EmailAddress	Repräsentiert eine Email-Adresse
Html	Repräsentiert eine HTML-Datei
MultilineText	Repräsentiert mehrzeiligen Text
Password	Repräsentiert ein Passwort
Text	Repräsentiert Text
Time	Repräsentiert eine Uhrzeit
Url	Repräsentiert eine URL

Model mit Data Annotations

Author
- FirstName : string
- Email : string
- Descript : string

```
public class Author
{
     [Display(Name="Ihr Name")]
     public string Name { get; set; }

     [DataType(DataType.EmailAddress)]
     public string Email { get; set; }

     public string Descript { get; set; }
}
```

Hands On

Data Annotation für Anzeige

Validierung

Data Annotations

- Steuern die Anzeige und die Validation von Modeleigenschaften (Properties)
- System.ComponentModel.DataAnnotations
- Built-In-Data-Annotations für Anzeige

Anzeige Attribute	Beschreibung
DataType	Legt den Datentyp fest
Display	Legt den Anzeigenamen fest
DisplayFormat	Legt fest wie Datenfelder dargestellt und formatiert werden
UIHint	Bestimmt das Control für die Modeleigenschaft

Model mit Data Annotations

Autor
- AutorId : int
- Name : string
- Beschreibung : string
- Geburtsdatum: DateTime

```
public class Autor
{
 public int AutorId { get; set; }

 [Display(Name="Name des Autors")]
 public string Name { get; set; }
 public string Beschreibung {get;set;}

 [DataType(DataType.Date)]
 public DateTime? Geburtsdatum {get;set;}
}
```

Model - Validierungsmöglichkeiten

- 5 Möglichkeiten der Model-Validierung
  - Built-In-Attribute
  - Benutzerdefinierte Attribute
  - Selbstvalidierendes Model
  - Clientseitig mit JQuery Validator Library
  - Remote

Built-In-Validierungsattribute

Validation Attribute	Beschreibung
Compare	Prüft, ob zwei Eigenschaften denselben Wert haben. (z.B. Passwort)
CreditCard	Prüft, ob der zu validierende Wert eine Kreditkartennummer ist
EmailAddress	Prüft auf Email-Format
MaxLength	Prüft auf eine maximale Länge
MinLength	Prüft auf eine minimale Länge
Range	Prüft, ob sich der zu validierende Wert in einem bestimmten Wertebereich befindet
RegularExpression	Validiert die Eigenschaft mit regulären Ausdrücken
Required	Markiert die Eigenschaft als Pflichtfeld

Model mit Validierungsattribute

Autor
- AutorId : int
- Name : string
- Beschreibung : string
- Geburtsdatum: DateTime

```
public class Autor
{
 public int AutorId { get; set; }

 [Required(ErrorMessage="Bitte …")]
 [StringLength(50)]
 [Display(Name="Name des Autors")]
 public string Name { get; set; }

 [StringLength(50)]
 public string Beschreibung {get;set;}

 [DataType(DataType.Date)]
 public DateTime? Geburtsdatum {get;set;}
}
```

Benutzerdefinierte Validierungsattribute

- Ableitung von Basisklasse **ValidationAttribute** und überschreiben von "**`IsValid`**" Methode

```
public class NoAdminAttribute : ValidationAttribute
{
  protected override ValidationResult IsValid(object value,
                                      ValidationContext validationContext)
  {
   string name = (string) value;
   if(name.ToLower() != "admin" && name.ToLower() != "administrator")
     {
       return ValidationResult.Success;
     }
     var error = "Der Wert darf nicht Admin oder Administrator sein";
     return new ValidationResult(error);
  }
}
```

Benutzerdefinierte Validierungsattribute

Autor
- AutorId : int
- Name : string
- Beschreibung : string
- Geburtsdatum: DateTime

```
public class Autor
{
 public int AutorId { get; set; }

 [Required(ErrorMessage="Bitte …")]
 [StringLength(50)]
 [NoAdmin]
 [Display(Name="Name des Autors")]
 public string Name { get; set; }

 [StringLength(50)]
 public string Beschreibung {get;set;}

 [DataType(DataType.Date)]
 public DateTime? Geburtsdatum {get;set;}
}
```

Serverseitige deklarative Validierung

- Überprüfen, ob Model korrekt validiert wurde mit Eigenschaft **`ModelState.IsValid`** und `optional` **`Fehlermeldung`**

```
public IActionResult Post(Autor autor)
{
    if(!ModelState.IsValid)
    {
      ModelState.AddModelError("Name","So ein Pech auch");
         return BadRequest(ModelState);
    }
    return View(autor);
}
```

Selbstvalidierendes Model

- Vorteil von vereinten Validierungen in einer Klasse
- Nachteil von nicht mehrfach verwendbaren Validierungen
- Schnittstelle **`IValidateObject`** muss implementiert werden

Selbstvalidierendes Model

```
public class Teilnehmer : IValidateObject
{
     …
     [StringLength(50)]
     //[NoAdmin]
     public string Nachname { get; set; }
     …
}

public IEnumerable<ValidationResult> Validate( ValidationContext
                                                        validationContext)
{
    List<ValidationResult> errors = new List<ValidationResult>();
    if (Nachname.ToLower() == "admin” || Nachname.ToLower() ==
                                                        "administrator”)
    {
        errors.Add(new ValidationResult("Der Nachname darf nicht Admin
          oder Administrator sein"));
    }
    return errors;
}
```

Clientseitige Validierung

- Notwendige Jquery-Bibliotheken
  - jquery-{version}.js
  - jquery.validate.js
  - jquery.validate.unobtrusive

- Normalerweise in Visual Studio Projektvorlage schon vorhanden

Remotevalidierung

- Kombination aus clientseitiger Validierung mit serverseitiger Validierung
- Validierung via AJAX-Anfrage
- NuGet Paket **Microsoft jQuery Unobtrusive Ajax** benötigt
  - jquery.unobtrusive-ajax.js

Remotevalidierung

- Erstellen einer Controller Action, die **JsonResult** zurückgibt

```
public JsonResult IsEmailUnique(string email)
{
    if(TeilnehmerAusDatenbank.Count(c => c.Email == email) == 0)
        return Json(true); //OK Email noch nicht vorhanden
    else
        return Json(false); //Fehler Email schon vergeben
}

public class Teilnehmer
{
      [Remote("IsEmailUnique","ControllerName", ErrorMessage="…")]
      public string Email { get; set;)
}
```

Hands On

Validierung

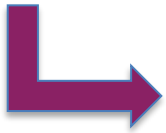
HTML Helper Methoden

- Unterstützen das Rendern von Html-Elementen in Views
- Diverse Built-in Helper
- Nutzen die Modelattribute für
  - Namensanzeige
  - Formatierung
  - Eingabeelemente
- Erstellen von eigenen HtmlHelper

Built-In Action Helper

- Html.ActionLink()

```
@Html.ActionLink("Klicken Sie hier", "Anzeige",new { id = 1})
```



```
<a href="/Home/Anzeige/1>Klicken Sie hier</a>
```

- Url.Action()

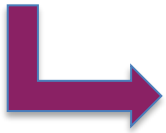
```
<img src="@Url.Action("GetBild", new { id = 1}) />
```

```
<img src="/bild/getbild/1" />
```

Built-In Display Helper

- Html.DisplayNameFor()

```
@Html.DisplayNameFor(model => model.FullName)
```



```
Der vollständige Name
```

- Html.DisplayFor()

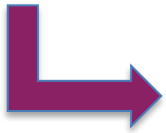
```
@Html.DisplayFor(model => model.FullName)
```

```
Max Mustermann
```

Built-In Editor Helper

- Html.LabelFor()

```
@Html.LabelFor(model => model.FullName)
```



```
<label for="FullName">Full Name </label>
```

- Html.EditorFor()

```
@Html.EditorFor(model => model.IsAdministrator)
```

```
<input type="checkbox" name="IsAdministrator">
```

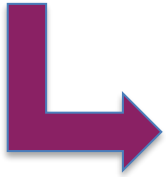
Speziellhelper EditorFor

Control	Model Klasseneigenschaft	Gerendertes HTML von EditorFor
TextBox	`public string Titel { get; set; }`	`<input type="text" name="Titel">`
Mehrzeilige Textbox	`[DataType(DataType.MultilineText)] public string Beschreibung { get; set; }`	`<textarea name="Beschreibung" rows="5" cols="2">`
Checkbox	`public bool IstAdministrator { get; set; }`	`<input type="checkbox" name="IstAdministrator">`

Built-In Begin Form Helper

- Html.BeginForm()

```
@using(Html.BeginForm("Erstelle","Kunde",FormMethod.Post,
new { enctype = "multipart/form-data"}))
{
  … Eingabefelder …
}
```

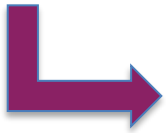


```
<form action="/Kunde/Erstelle" method="post"
                    enctype="multipart/form-data>
</form>
```

Built-In Validation Helper

- Html.Validation()

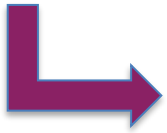
  @Html.ValidationSummary()



```
<ul>
  <li>Bitte geben Sie einen Nachname ein</li>
  <li>Bitte geben Sie eine gültige Emailadresse ein</li>
</ul>
```

- Html.ValidationMessageFor()

  @Html.ValidationMessageFor(model => model.Email)



Bitte geben Sie ein gültige Emailadresse ein

Custom Razor-Helper

- Helper sind Methoden um wiederkehrende Aufgaben auszulagern

```
@helper RenderFirmenLogo(string logoName)
{
    <img src="@logoName" width="150" height="150" />
}

<div class="logo">
     @RenderFirmenLogo("ilci.png")
</div>
```

Html Helper Extension Method

- Erweiterung der HtmlHelper-Klassen durch selbstdefinierte Helper-Klasse

```
public static class HtmlHelperExtensions
{
  public static IHtmlContent Truncate(this IHtmlHelper helper,
  string text, int length = 20)
  {
     if (text.Length > length)
           text = text.Substring(0, length) + "...";

     return new StringHtmlContent(text);
  }
}
```

Hands On

Html Helper

TagHelper

- Was sind TagHelper?
- Built-In TagHelper

Was sind TagHelper?

- TagHelper sind Klassen, die Html-Elemente verändern, um sie z.B. mit zusätzlichen Inhalten zu ergänzen oder sie mit neuem Inhalt zu ersetzen (ähnlich Angular-Direktiven)

- In der _*ViewImports.cshtml* müssen sie registriert werden (Assembly Name)
  - @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

- Zwei Methoden zum Überschreiben **Process** & **ProcessAsync**

- Deaktivierung für individuelle Elemente
  - <!span asp-validation-for..></!span>

- @tagHelperPrefixe möglich

Vergleich

HtmlHelpers

@Html.Label(„Vorname“, new { @class = „caption“})

WebControls

<asp:Label runat=„server“ ID=lblVorname“></asp:Label>

TagHelpers

<label class=„caption“ asp-for=„Vorname“></label>

Wie werden TagHelper aktiviert?

- AttributName
- Html Element Tag
- Eltern Element

TagHelper Process Methode

- Razor View Engine sucht nach validen Tag Helpern
  - <my-custom info=„Bill“></my-custom>

- Führt die Process Methode aus

```
public class MyCustomTagHelper : TagHelper
{
  public string Info { get; set; }

  public override void Process(TagHelperContext context, TagHelperOutput output)
  {}
}
```

Verbindung von Html Element Attributen zu C# Properties

- Html Attribute

```
<auto model=„911“ marke=„porsche“></auto>
```

- C# Property

```
public class AutoTagHelper : TagHelper
{
        public string Model { get; set; }
        public string Marke { get; set; }

       //alt. Syntax mit Namen des Html-Attributes
       [HtmlAttributName(„model-name“)]
       public string Model { get; set; }
}
```

Hands On

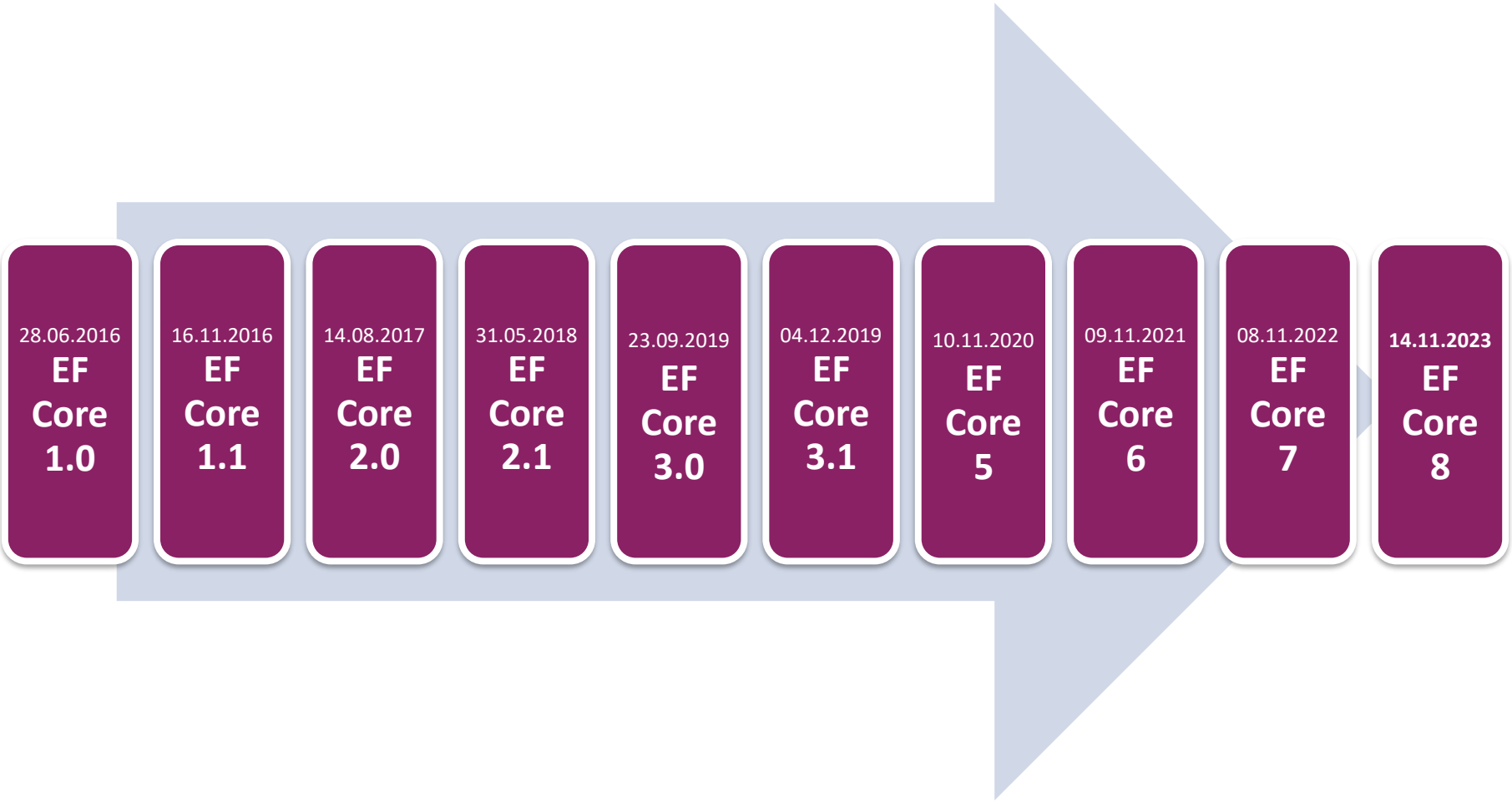
TagHelper

Exkurs: Entity Framework Core 8

ADO.NET Entity Framework

- ORM-Framework
  - Bildet Objekte auf eine relationale DB ab
  - Web Forms, MVC, WPF, WCF, Web API
  - Unabhängig von verwendeter Datenbank

Versionshistorie Entity Framework



Ohne Entity Framework

- Fehleranfällig, Unsicher, Kompliziert

```
string con = "Data Source=.\\;Initial Catalog=AutorDB;IntegratedSecurity=True";
string cmd= "INSERT INTO Author (LastName,FirstName) VALUES (‘Twain', ‘Mark')";

using (SqlConnection connection = new SqlConnection(connString))
{
   using(SqlCommand com= new SqlCommand(cmdString))
   {
      com.Connection= con;
      connection.Open();
      com.ExecuteNonQuery();
   }
}
```

Mit Entity Framework

- Objekte <> Tabellen Abbildung
- Schutz vor SQL-Injections
- Vermeidung von Syntaxfehlern
- ConnectionString in Konfigurationsdatei

```
using (var db = new AuthorDBContext())
{
   db.Authors.Add(new Author { LastName = "Twain", FirstName = "Mark" });
   db.SaveChanges();
}
```

EF Begrifflichkeiten

- Domain-Model
  - Konzeptionelles Model, welches ein Geschäftsproblem darstellt, aber noch keine technische Lösung
- Objekt – Modell
  - Eine Klasse, die die Eigenschaften eines realen oder abstrakten Objektes beschreibt
- ER – Modell
  - Visualisiertes Datenmodell
- DB – Modell
  - Datenbankdiagramm einer physischen Datenbank

Aufbau Entity Framework

- Entity
  - = Tabelle in Datenbank
  - Eigenschaften = Spalten der Tabellen
- Relationship
  - Fremdschlüsselbeziehungen in der Datenbank
- Context
  - Repräsentiert den Zugriff auf die Datenbank
  - CRUD

Model Workflows EF Core

Vorgehensweise	Datenbank vorhanden	Beschreibung
Code First	**Nein**	Objekte und deren Beziehungen werden in Klassen beschrieben. Daraus wird dann mit dem EF die DB erstellt
Reverse Engineering Code First	**Ja**	Generierung von Entity Framework Klassen aus der DB

Installation des Entity Framework Core

- NuGet-Paket-Manager
  - Ab VS 2012 standardmäßig installiert
  - Als Erweiterung für VS 2010 verfügbar
- NuGet-Konsole
  - Install-Package EntityFrameworkCore

Code First Entitäten

- Für jede Entity eine Klasse erstellen

```
public class Author
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public virtual List<Quote> Quotes { get; set; }
}
```

```
public class Quote
{
    public Guid Id { get; set; }
    public string QuoteText { get; set; }
    public int AuthorId { get; set; }
    public virtual Author Author { get; set; }
}
```

Beziehung 1 : n

Code First Data Context

- Erstellen eines EF Data Context
- Data Context => DB
- Jede Entity **T** eine Eigenschaft **DbSet<T>** hinzufügen

```
public class QuoteContext : DbContext
{
   public QuoteContext(DbContextOptions<QuoteContext> options) :
                                base(options)
   {}

   public DbSet<Author> Authors => Set<Author>();

   public DbSet<Quote> Quotes => Set<Quote>();
}
```

Verwenden EF in Controller

- DataContext instanziieren
- Gewünschte Entitäten abfragen (Filtern möglich)
- Als Model an View übergeben

```
public class AuthorsController
{
   private QuoteContext db = new QuoteContext();

   public IActionResult GetAuthors()
   {
       var authors = db.Authors.ToList();
       return View(authors);
   }
}
```

Database Migrations

- Löst das Problem Code und Datenbank synchron zu halten
- Möglichkeit von Snapshots
  - Generierung von Scripten um Code/Datenbank zu synchronisieren
  - Migrierung & Scripting SQL diverser Snapshots
- NuGet-Konsole
  - **`Add-Migration`** fügt neue Migration ein
  - **`Update-Database`** synchronisiert Modell mit Datenbank
  - **`Script-Migration`** erstellt SQL-Script

EF Core Commands

Installation des Tools
dotnet tool install --global dotnet-ef

https://docs.microsoft.com/de-de/ef/core/miscellaneous/cli/dotnet

Package Manager Console	dotnet CLI	Beschreibung
add-migration <migrationsname>	**dotnet ef migrations add <migrationsname>**	Erstellt eine Migration mit Migrations Snapshot
Remove-migration	**dotnet ef migrations remove**	Entfernt die letzte hinzugefügte migration
Update-database	**dotnet ef database update**	Aktualisiert die
Script-migration	**Dotnet ef migrations script**	Generiert ein SQL Script mit allen Migrationen

Layout

Layout - Views

- Vorlage für mehrere Webseiten (Masterlayout)
- Webseite kann mehrere Masterlayouts besitzen
- Standardmasterlayout für die Views wird in der **`ViewStart.cshtml`** festgelegt oder individuell in der View
- Standardlayout => Views/Shared/_Layout.cshtml
- Twitter Bootstrap Standard

_Layout.cshtml

```
<!DOCTYPE html>
<html>
  <head>
  <meta name= "viewport" content= "width=device-width" />
  <title>@ViewData["Title"]</title>
   </head>
  <body>
    <div>
      <h3>Views / Shared / Layout.cshtml</h3>
      @RenderBody()
    </div>
    @RenderSection("scripts", false)
  </body>
</html>
```

_Layout.cshtml - Elemente

Element	Beschreibung
@ViewBag.Title bzw. @ViewData[„Title“]	Titel der individuellen Content-Views
@RenderBody	Rendering der Content-View (darf nur einmal vorhanden sein)
@Render.Section	Aufteilung von inhaltlich zusammenhängenden Layout-Regionen

Partial Views

- Templates aus HTML und Razor-Code
- Können Bestandteile von Layout-Views, Standard-Views und partiellen Views sein
- Zweck ist die Extrahierung von Funktionalitäten wegen Übersicht, Austauschbarkeit und Wiederverwendung
- Möglichkeit der Controllerzuordnung (im Controller-Ordner) oder Anwendungszuordnung (Shared-Ordner)
- Konvention: vorangehender Underscore im Namen

Partial Views

- Mit HTML Helpers werden partielle Views aufgerufen
  - **@Html.Partial** für partielle Views mit gleichem Eltern Model
  - **@Html.Action** für partielle Views mit unterschiedlichem Eltern Model
  - @{await **Html.RenderPartialAsync**("_AuthorsList");}
  - <**partial** name="_AuthorsList" model="Model" />

- ViewBag/ViewData um Daten zwischen Controller Aktion, Eltern-View und Partielle-View zu teilen

Mobile Views

- Verschiedene Möglichkeiten Mobile Views zu erstellen
  - CSS MediaQueries (manuell)
  - Bootstrap (Framework)
  - JQuery Mobile

ViewComponents

ViewComponents

- ViewComponents sind Klassen ähnlich PartialViews die Fragmente von Html-Ausgaben darstellen. Es sind wiederverwendbare Webbausteine
- Sie besitzen eine Elternseite bzw. werden von ihr aufgerufen
- Sie besteht aus einer Klasse abgeleitet von *ViewComponent* und normalerweise einer View
- Typische Anwendungsfälle
  - Tag Cloud, Einkaufwagen, letzter Blogartikel

ViewComponents

- Suffix ViewComponent
- Unterstützt Constructor DI
- Kein Bestandteil des Controllerlebenszyklus
  und somit keine Filterverwendung möglich
- Invoke bzw. InvokeAsync – Methode, die ein IViewComponentResult zurückgibt
- Keine Modellbindung
- Pfade
  - /Pages/Components/<viewname>
  - Views/<controllername>/Components/<ansichtskomponentenname>/<ansichtsname>
  - Views/Shared/Components/ /<ansichtskomponentenname>/<ansichtsname>

ViewComponents

- Aufruf durch

  @await Component.InvokeAsync(„VIEW NAME“, anonymous type)

- Alternative

  <**vc:**[view-component-name]></vc:[view-component-name]>

Hands On

ViewComponents

Bootstrap 5

Die „K“-Frage für Webentwickler

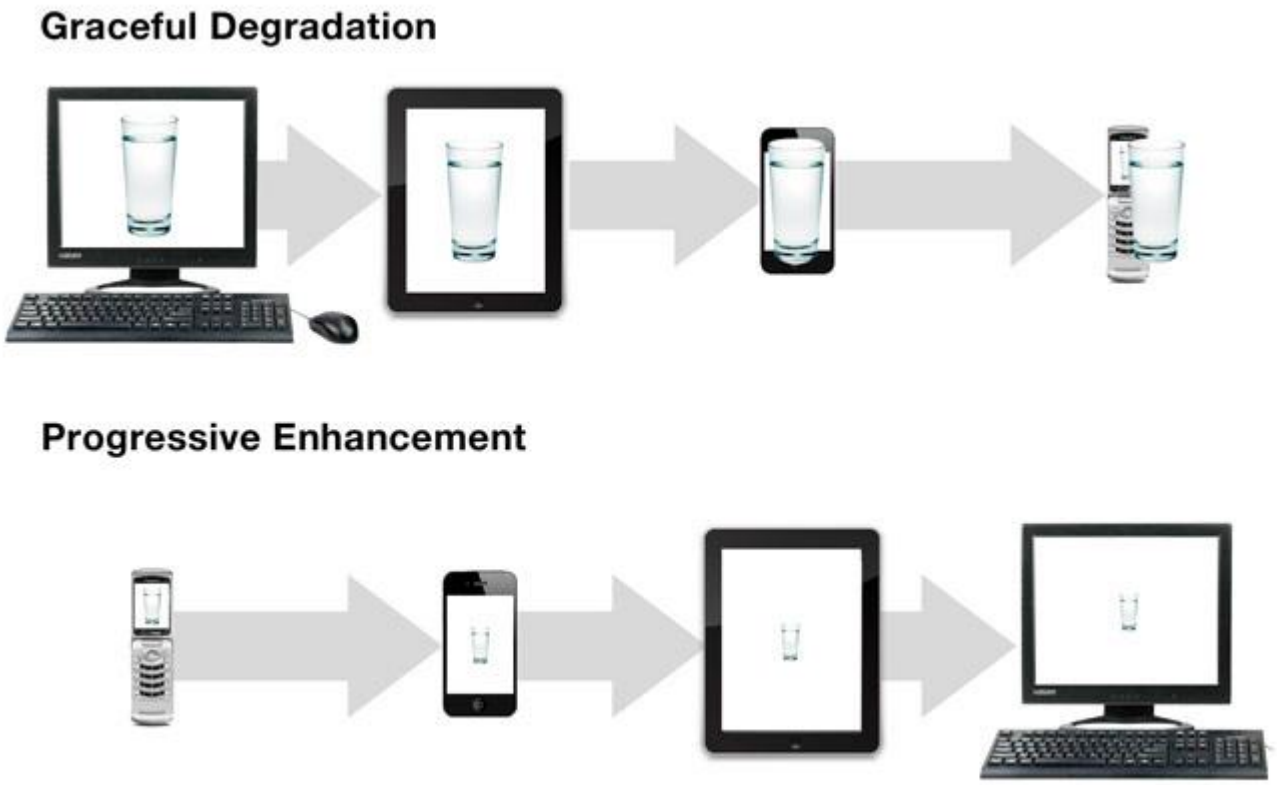
Künstler

Klempner

Was ist Bootstrap?

- Open Source Front-End Framework von Twitter
- Mobile First Responsive Web Design
- Modular
- Besteht aus CSS und JavaScript
- Vielfalt an Designkomponenten
- Standard-Designvorlage seit MVC 5

Mobile First Responsive Design

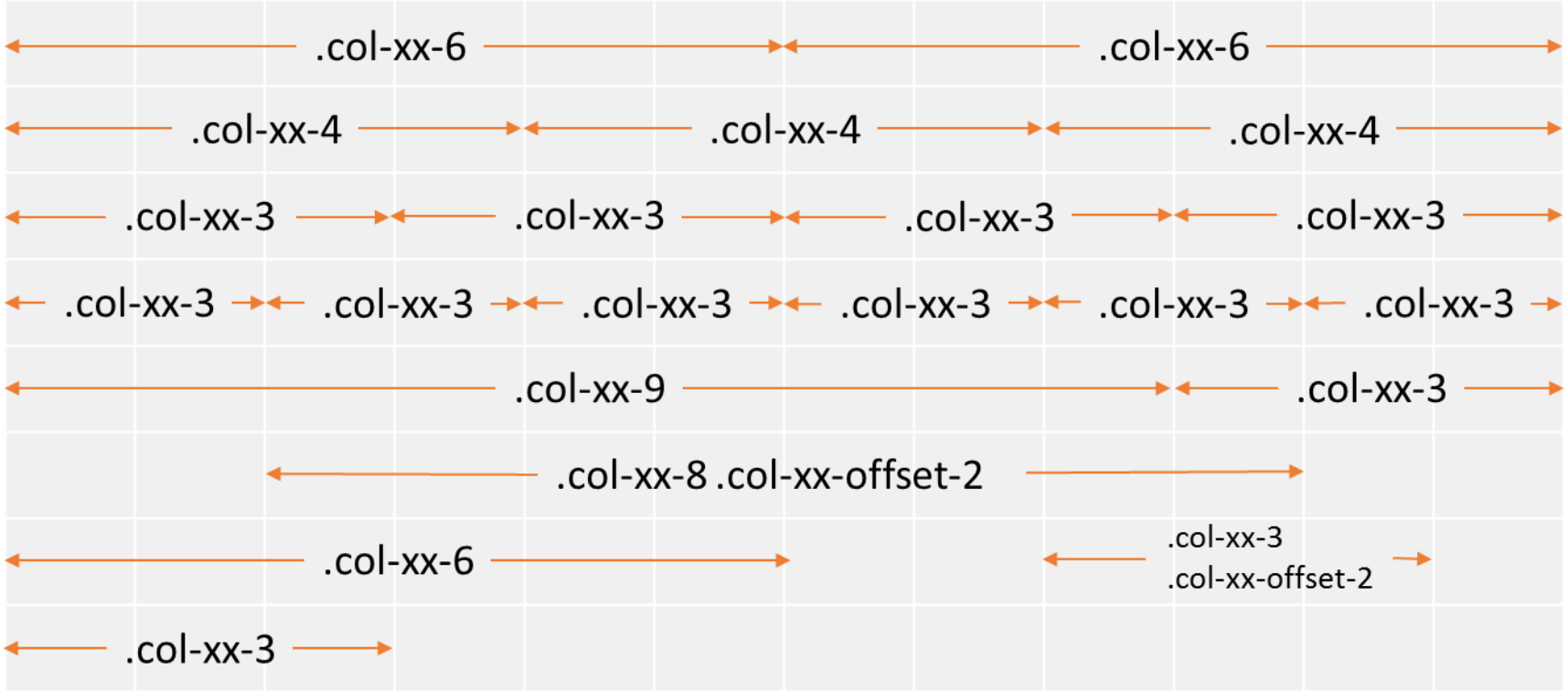


Quelle: http://www.deepblue.com

Grid System

	xs <576px	sm ≥576px	md ≥768px	lg ≥992px	xl ≥1200px	xxl ≥ 1400
Max container width	None (auto)	540px	720px	960px	1140px	1320px
Class prefix	**.col-**	**.col-sm-**	**.col-md-**	**.col-lg-**	**.col-xl-**	**.col-xxl-**
# of columns	12					
Gutter width	1.5rem (.75rem on left and right)					
Custom gutters	Yes					
Nestable	Yes					
Column ordering	Yes					

12 Column Grid System



Bootstrap Komponenten Auszug

Default Primary Success Info Warning Danger

« 1 2 3 4 5 »

300x200

Thumbnail label

Cras justo odio, dapibus ac facilisis in, egestas eget quam. Donec id elit non mi porta gravida at eget metus. Nullam id dolor id nibh ultricies vehicula ut id elit.

Button Button

300x200

Thumbnail label

Cras justo odio, dapibus ac facilisis in, egestas eget quam. Donec id elit non mi porta gravida at eget metus. Nullam id dolor id nibh ultricies vehicula ut id elit.

Button Button

300x200

Thumbnail label

Cras justo odio, dapibus ac facilisis in, egestas eget quam. Donec id elit non mi porta gravida at eget metus. Nullam id dolor id nibh ultricies vehicula ut id elit.

Button Button

Collapsible Group Item #1

Anim pariatur cliche reprehenderit, enim eiusmod high life accusamus terry richardson ad squid. 3 wolf moon officia aute, non cupidatat skateboard dolor brunch. Food truck quinoa nesciunt laborum eiusmod. Brunch 3 wolf moon tempor, sunt aliqua put a bird on it squid single-origin coffee nulla assumenda shoreditch et. Nihil anim keffiyeh helvetica, craft beer labore wes anderson cred nesciunt sapiente ea proident. Ad vegan excepteur butcher vice lomo. Leggings occaecat craft beer farm-to-table, raw denim aesthetic synth nesciunt you probably haven't heard of them accusamus labore sustainable VHS.

Collapsible Group Item #2

Collapsible Group Item #3

Home 42 Profile Messages 3

Tooltip on right Tooltip on right

Home Profile Messages

Well done! You successfully read this important alert message.

Heads up! This alert needs your attention, but it's not super important.

Warning! Better check yourself, you're not looking too good.

Oh snap! Change a few things up and try submitting again.

Modal title ×

One fine body…

Close Save changes

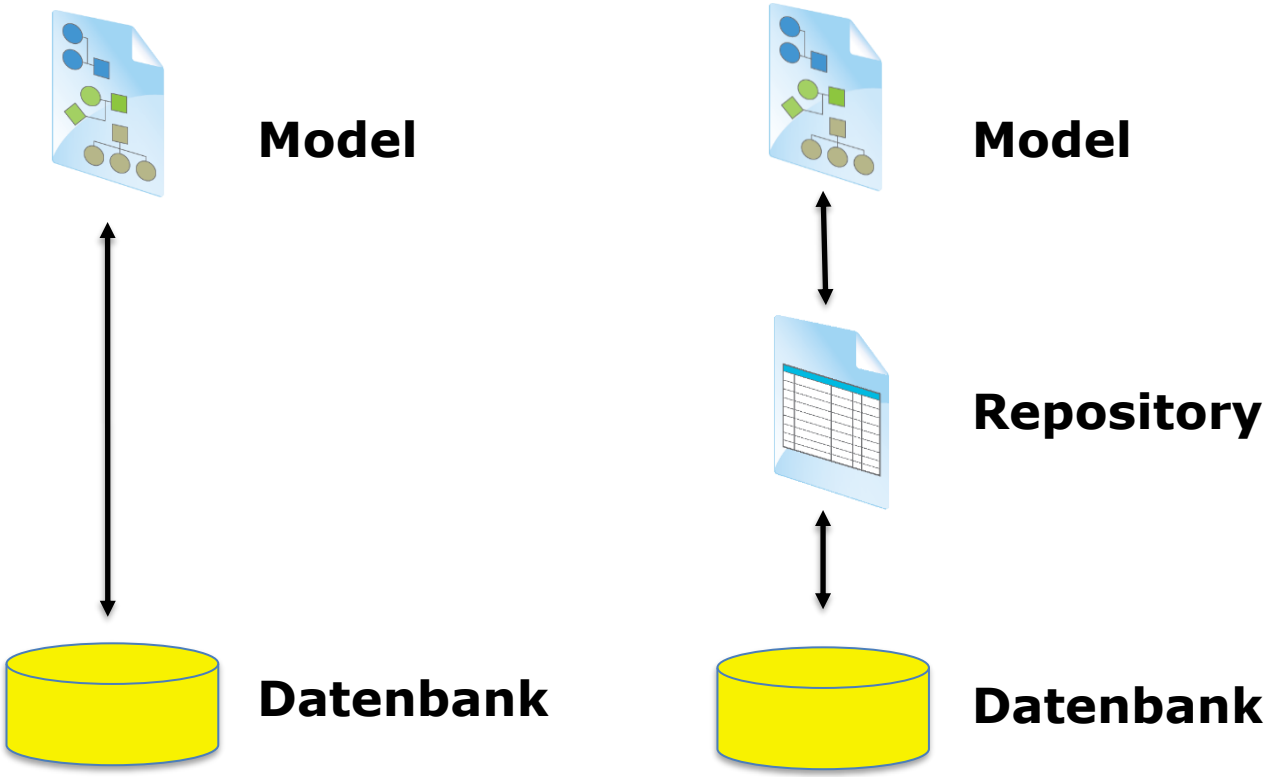
Links

- Bootstrap 5
  - http://getbootstrap.com/
- Themes
  - http://bootswatch.com/ (Free)
  - https://wrapbootstrap.com/ (Kommerziell)
- Online Editor
  - http://bootply.com/
- Fuel UX
  - http://earmbrust.github.io/dieselui/#

Repository

Repository

- Separation of Concerns
- Lose Kopplung



Dependency Injection

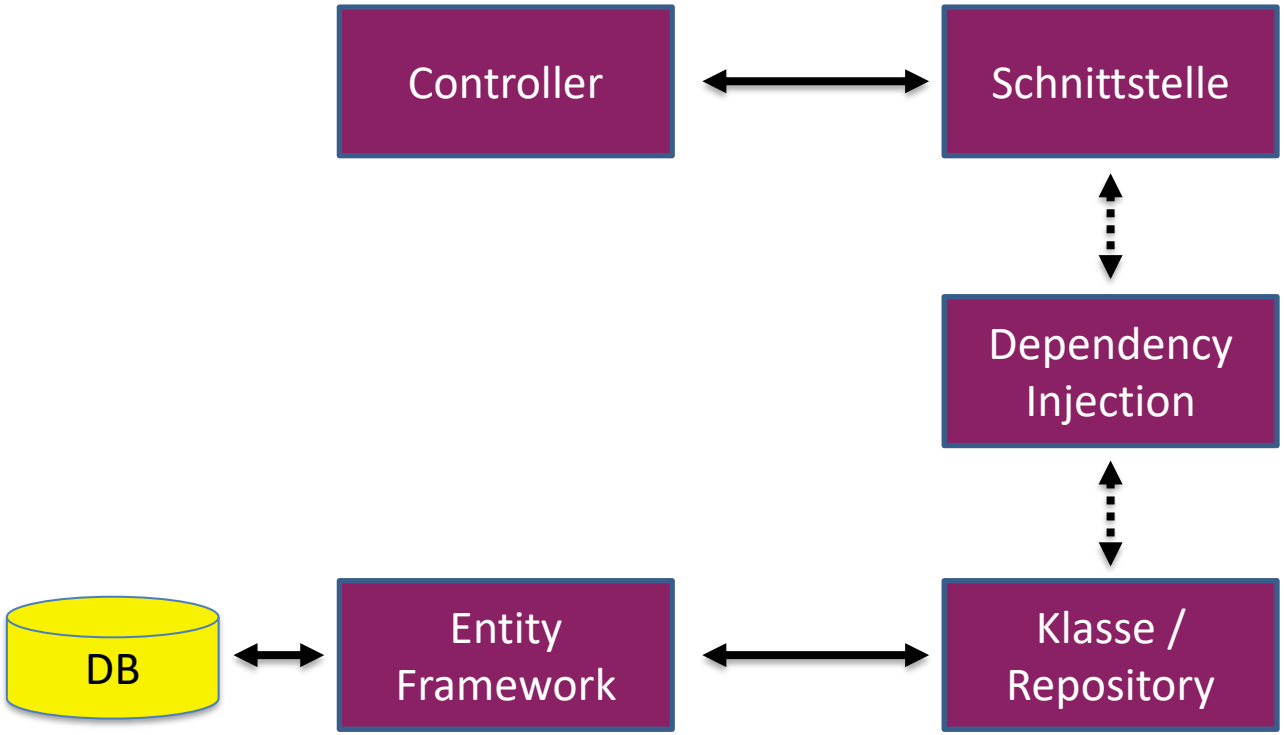
IoC

IoC / Dependency Injection

- ***I**nversion **o**f **C**ontrol* delegates the function of selecting a concrete implementation type for a class‘s dependencies to an external component

- *Dependency Injection* is a specialization of the IoC Pattern. The Dependency Injection pattern uses an object – the container – to initialize objects and provide the required dependencies to the object

Zugriff mit Dependeny Injection



Dependency Injection Beispiel

```
public class AuthorsController : Controller
{
  private readonly IAuthorRepository _authorRepository;

  protected AuthorController(IAuthorRepository authorRepository)
  {
    _authorRepository = authorRepository;
  }
}
```

Interface, *keine* konkrete Implementierung

Konstruktor Injection

Dependency Injection

- Ziel ist die Flexibilität und Testbarkeit zu steigern
- Erleichtert das Austauschen & Testen von Abhängigkeiten
- Built-In-ASP.NET Core
- Populäre DI-Frameworks
  - Microsoft Unity
  - Ninject
  - StructureMap

DI Scopes

- BuilderServices in Program verantwortlich für die Bereitstellung der Services

Scope	Bedeutung	Erläuterung
Transient	**vorübergehend**	Eine neue Instanz des Typs wird bei jeder Anforderung des Typs verwendet.
Scoped	**bereichsbezogen**	Eine neue Instanz des Typs wird bei seiner erstmaligen Anforderung in einer bestimmten HTTP-Anforderung erstellt und anschließend für alle nachfolgenden Typen verwendet, die während dieser HTTP-Anforderung aufgelöst werden
Singleton	**übergreifend**	Eine Instanz des Typs wird ein Mal erstellt und von allen nachfolgenden Anforderungen für diesen Typ verwendet.

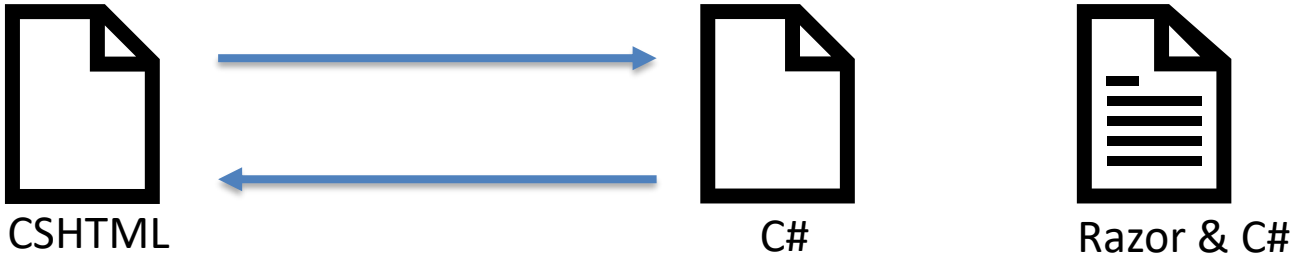
Hands On

DI

Razor Pages

Razor Pages

- Bieten eine vereinfachte Seitenstruktur ohne MVC-Pattern mit der Razor Syntax
- Design Pattern
  - CSHTML = Razor und HTML Markup
  - C# = Code und Logik
- Standardordner **Pages**
- Html, CSS und C# Kenntnisse erforderlich



Unterschiede ASP.NET Core Razor Pages & MVC

Razor Pages	MVC
Page zentriert	Action zentriert
Einfachere Struktur	Komplexere Struktur
Viele Gemeinsamkeiten mit Blazor	

Razor Page & Model

```
@page

@model IndexModel

<div>
  <p>@Model.Email<</p>
</div>
```

Page Direktive erforderlich

Model Direktive verweist auf Klasse

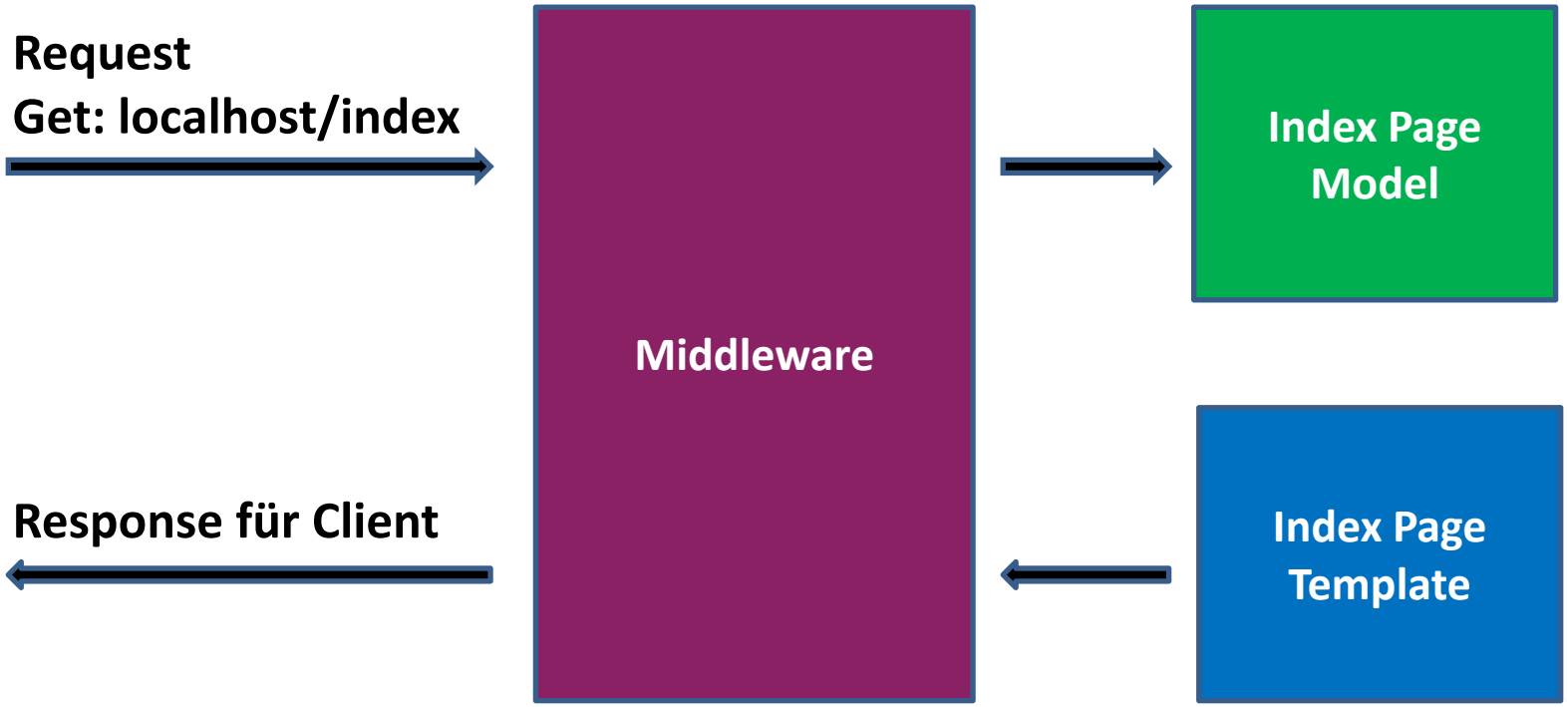
Model Eigenschaften Zugriff

```
class IndexModel : PageModel
{
   public Email {get; set;}
}
```

Erbt von PageModel

Eigenschaft zugreifbar von der Razor Seite

Razor Page Rquest Pipeline



Page Model Request Handling

HTTP Request Type	Page Model Convention	Beschreibung
GET	OnGet()	Seite laden
POST	OnPost()	Erstellung neue Daten
PUT	OnPut()	Änderung Daten
DELETE	OnDelete()	Löschen Daten

Model Binding

Model Binding

Query Parameter

GET localhost/autor**?orderby=desc**

→ Werte aus URL

Key/Value Form Daten

POST localhost/autor
name=Ilci&beschreibung=Dozent

→ Formular Werte

JSON/XML Daten

POST localhost/autor
{
„name“:“Einstein“,
“beschreibung“:“Physiker“
}

→ Datenblock

Hands On

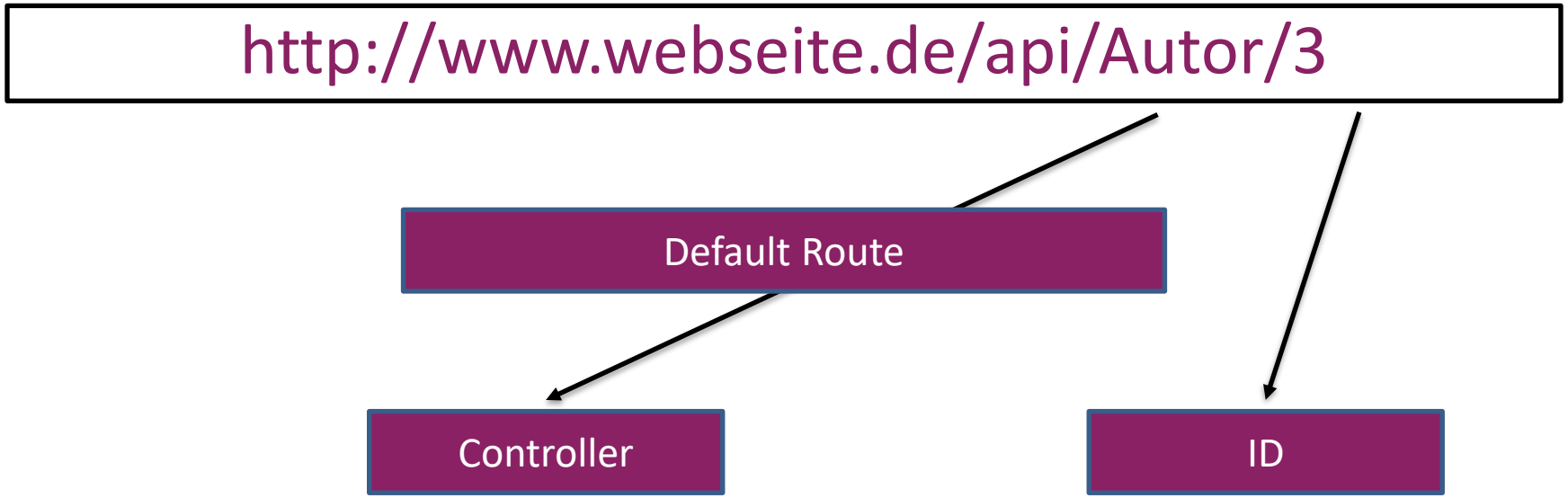
RazorPages

Routing

Routing

- ASP.NET Routing Engine
- Hinzufügen & Konfigurieren von Routen
- Parameterübergabe mit Routen

Die Standardroute



- Benutzerdefinierte Routen
  - Einfache URLs / Muss nicht Projekt-Struktur entsprechen
  - Nützlich für SEO

Routing-Konfiguration

- Routing wird in **`Startup.cs -> Configure`** festgelegt
- Reihenfolge der Abarbeitung wichtig! Wer zuerst kommt, malt zuerst
- Beim Suchen der passenden Route wird die Anzahl der Abschnitte in der URL gezählt und mit der in den Routen verglichen
- Route beinhaltet Name, URL, Constraints and Defaults

Konventionell Routing-Beispiel

Standardroute

```
endpoint.MapRoute(
    name: „default",
    template: "{controller=Home}/{action=Index}/{id?},
);
```

Benutzdefinierte Route

```
routes.MapRoute(
    name: "Authors",
    url: "/authors/authorname/{name}",
    defaults: new { controller = "Authors" },
    constraints: new { name = @"[a-zA-Z ]+$" });
```

Attribut Routing

- Vereinfacht das Routing
- Empfohlen für ASP.NET Core WebAPI
- Features:
  - Route Constraints
    - Custom Route Constraints
  - Route Names

Attribut Routing Beispiel

```
[Route("autors")]
public class AuthorsController : Controller
{
  [Route] // z.B ./autors
  public IActionResult Get() { return Ok(authors); }

  [Route("{id}")] // Opt. Para. z.B /autors/5
  public IActionResult Get(int id) { return Ok(author);}

  [Route("authorname/{name}", Name = "GetAuthorByName")]
  // z.B /authors/authorname/Albert%20Einstein
  public IActionResult GetAuthorByName(string name)
  { return Ok(); }
}
```

Attribut Routing Constraints

Constraint	Beschreibung	Beispiel
alpha	Matches uppercase or lowercase Latin alphabet characters (a-z, A-Z)	{x:alpha}
bool	Matches a Boolean value.	{x:bool}
datetime	Matches a **DateTime** value.	{x:datetime}
decimal	Matches a decimal value.	{x:decimal}
guid	Matches a GUID value.	{x:guid}
int	Matches a 32-bit integer value.	{x:int}
length	Matches a string with the specified length or within a specified range of lengths.	{x:length(6)} {x:length(1,20)}
max	Matches an integer with a maximum value.	{x:max(10)}
maxlength	Matches a string with a maximum length.	{x:maxlength(10)}
min	Matches an integer with a minimum value.	{x:min(10)}
minlength	Matches a string with a minimum length.	{x:minlength(10)}
range	Matches an integer within a range of values.	{x:range(10,50)}
regex	Matches a regular expression.	{x:regex(^\d{3}-\d{3}-\d{4}$)}

Custom Route Constraints

- Möglichkeit eigene Routing Constraints zu definieren für komplexere Logik
  1. Implementierung von `IRouteConstraint`
  2. Registrieren in **ConfigureServices**
  3. Action mit benutzerdefiniertem Attribut markieren

Custom Routing Contraints Beispiel

1. Implementierung

```
public class AuthorDescriptionConstraint : IRouteConstraint
{
    private readonly string[] _validAuthorDescription;
    public AuthorDescriptionConstraint(string options)
    {
        _validAuthorDescription = options.Split(',');
    }

   public bool Match(HttpContext httpContext, IRouter route, string routeKey,
                                RouteValueDictionary values, RouteDirection routeDirection)
   {
      object value;
      if (values.TryGetValue(routeKey,out value) && value != null)
      {
      return _validAuthorDescription.Contains(value.ToString(),StringComparer.OrdinalIgnoreCase);
      }
        return false;
   }
}
```

Custom Routing Contraints Registrierung

2. Registrieren

```
public void ConfigureServices(IServiceCollection services)
{
        …
        //Custom Route Constraints
        services.AddRouting(options =>
        {
        options.ConstraintMap.Add("authordescription",typeof(AuthorDescriptionConstraint));
        ….
        }
}
```

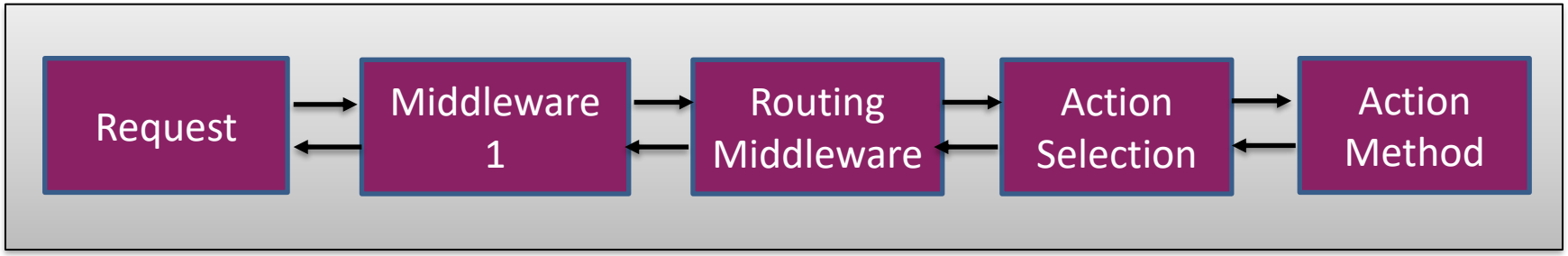
3. Action Attribute

```
[Route(„authordescription/{description:authorDescription(Dozent,IT-Trainer)}")]
public IActionResult ShowOnlyDozenten(string description)
{
    var authors = db.Authors.Where(c => c.Description.Contains(description));
    return Ok(autors);
}
```

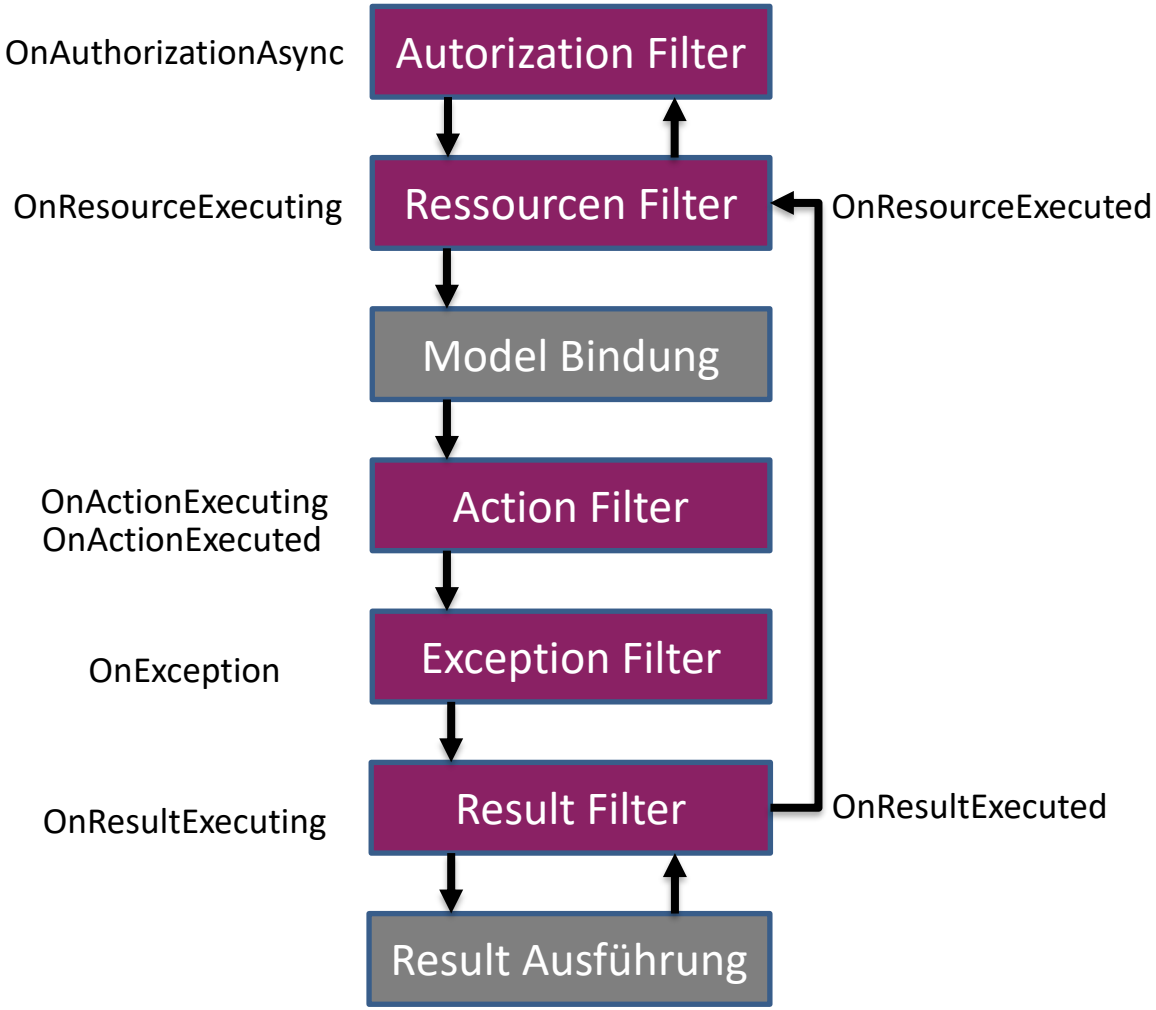
Filter

Filter

- Filter bieten eine Möglichkeit **vor** und **nach** dem Ausführen von Aktionen **vordefinierte** oder **benutzerdefinierte** Logiken/Methoden aufzurufen
- Filter könne auf Aktions-, Controller- und Anwendungs- Ebene sein
- 5 Built-in Filterarten



Reihenfolge Filterausführung



Built-In-Filterarten

Filtertyp	Schnittstelle	Methode	Beschreibung
Authorization	IAsyncAuthorization Filter	OnAuthorizationAsync	Ausführung vor dem Beginn einer Aktion zwecks Autorisierung
Resource	IResourceFilter/ IAsyncResourceFilter	OnResourceExecuting OnResourceExecuted	Werden am Anfang/Ende einer Anforderung aktiv, zum Bearbeiten einer Anforderung
Action	IActionFilter	OnActionExecuting OnActionExecuted	Ausführung vor/nach einer Aktion
Exception	IExceptionFilter/ IAsyncExceptionFilter	OnException	Ausführung, wenn eine Ausnahme im Controller auftritt
Result	IResultFilter / IAsyncResultFilter	OnResultExecuting OnResultExecuted	Ausführung vor/nach dem Aktionsergebnis

Action Filter Beispiel

```
public class SampleActionFilter : IActionFilter
  {
     …
     public void OnActionExecuting(ActionExecutingContext context)
     {
        //before action
        _logger.LogInformation($"Routing-Informationen: Controller ->
                                    {context.RouteData.Values["controller"]} " +
                                     $"## Action -> {context.RouteData.Values["action"]}");
     }

     public void OnActionExecuted(ActionExecutedContext context)
     {
        //after action
        _logger.LogInformation($"Erfolgreich ausgeführt");
     }
  }
```

Security Filters

- Authorize
  - Kontrolliert Wer den Controller/Action zugreifen darf via Eigenschaften Users / Roles
- ValidateAntiForgeryToken
  - Schützt vor cross-site request forgery (csrf)
  - Verlangt das Hinzufügen von Anti-Forgery Token zur View
- RequireHttps
  - Verlangt SSL

Hands On

Filter

Tracing / Logging

Tracing / Logging

- Built-In-Tracing/Logging durch ILoggerFactory der Microsoft.Extensions.Logging bzw. ILogger<T> mit DI
  - Aktivierung in `Configure`
    - `loggerFactory.AddConsole();`
    - `Feature Dependency Injection`
  - Benutzerdefiniertes Logging möglich durch `3rd Party-LoggingProvider`
- Loggingmöglichkeiten
  - NLog
  - Serilog

Hands On

Logging mit NLOG

Fehlerbehandlung

Fehlerbehandlung

- Verschiedene Fehlerbehandlungsmöglichkeiten
  - Action-Ebene (**`ExceptionFilterAttribute`**)
  - Controller-Ebene **`(ExceptionFilterAttribute)`**
  - Global durch **`IExceptionHandler oder ExceptionFilterAttribute`**
  - `Spezielle Entwickler-Exception-Middleware`
    - `Configure -> app.UseDeveloperExceptionPage();`

ExceptionFilterAttribute

```
public class ResourceRemovedAttribute : ExceptionFilterAttribute
{
  public override void OnException(ExceptionContext context)
  {
   if (context.Exception is ResourceRemovedException)
   {
     context.Result = new StatusCodeResult(StatusCodes.Status410Gone);
   }
  }
}
```

```
[ResourceRemoved]
public IActionResult Delete(int id)
{
   … auslösen der ResourceRemovedException
}
```

XSS – Cross-Site-Scripting

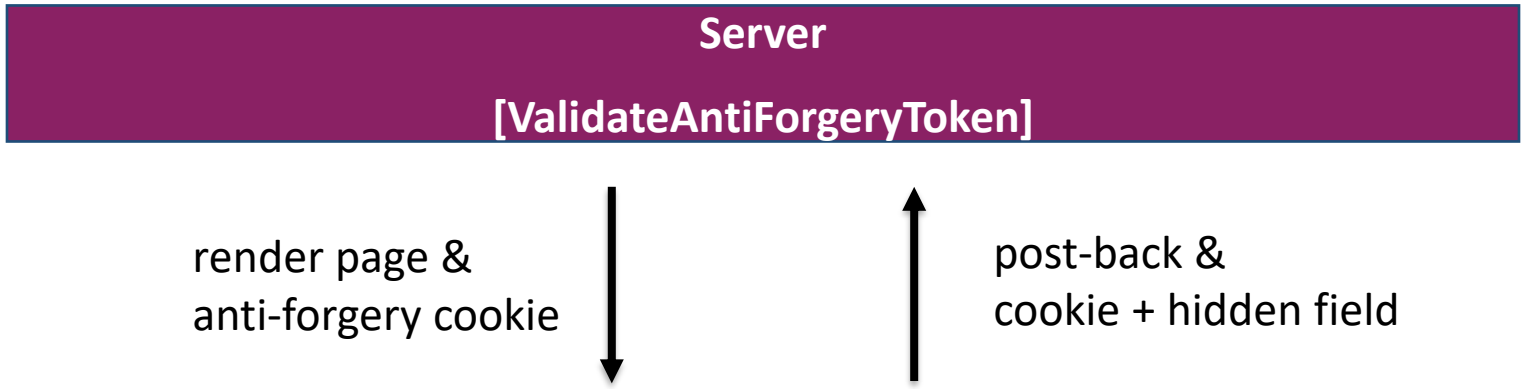
- Beruht auf dem Einschleusen von JavaScript Code in einer dynamischen Webanwendung
- Die Ursache ist das Zusammensetzen von Ausgaben aus Platzhalter und Benutzereingaben
- Wird verhindert indem die Eingabe/Ausgabe enkodiert (Standard in .NET 4.5.+)
- Für ältere Versionen AntiXSS-Library

XSS – Cross-Site-Scripting

- Beruht auf dem Einschleusen von JavaScript Code in einer dynamischen Webanwendung
- Die Ursache ist das Zusammensetzen von Ausgaben aus Platzhalter und Benutzereingaben
- Wird verhindert indem die Eingabe/Ausgabe enkodiert (Standard in .NET 4.5.+)
- Für ältere Versionen AntiXSS-Library

CSRF - Abwehr

- [ValidateAntiForgeryToken] für Action
- @Html.AntiForgeryToken() für View




```
<form>
   //@Html.AntiForgeryToken()
   <input type="hidden" value="anti-fogery token" />
    …
</form>
```

Unit Tests Definition

- Ein **Modultest** (auch **Komponententest** oder oft vom engl. **unit test** als **Unittest** bezeichnet) wird in der Softwareentwicklung angewendet, um die funktionalen Einzelteile (*Module*) von Computerprogrammen zu testen, d. h. sie auf korrekte Funktionalität zu prüfen.

  (Quelle: Wikipedia / https://de.wikipedia.org/wiki/Modultest)

Unit Test

- Verschiedene Phasen
  - Arrange (Vorbereitung)
    - Arrangiert Daten auf denen die Test später laufen
  - Act (Ausführung)
    - Ruft die Methoden auf, die getestet werden sollen
  - Assert (Ergebnis)
    - Überprüft, ob das Ergebnis den Erwartungen entspricht
- Visual Studio integriert Testmöglichkeit

Test Driven Development

Test schreiben

- Problem verstehen
- Gewünschtes Verhalten spezifizieren
- Test ausführen
- Test schlägt fehl

Test bestanden

- Code erstellen
- Test ausführen
- Test erfolgreich

Refactor

- Code aufräumen
- Test erfolgreich

Unit Tests mit ASP.NET Core MVC

- Neues Test-Projekt zur Solution hinzufügen
- Controller testen:
  1. Erstellen eines Repository Interface
  2. Implementieren des Repository Interface
  3. Implementierung eines Fake Repositories
  4. Benutzen des Fake Repositories um Controller zu testen

Mocking Frameworks

- Ein Mocking Framework automatisiert die Erstellung von Mock Objekten während der Test
- Hauptvorteil der Zeitersparnis beim Schreiben von Unit-Tests
- Verschiedene Frameworks
  - Moq
  - JustMock

Hands On

Unit Test / Mocking

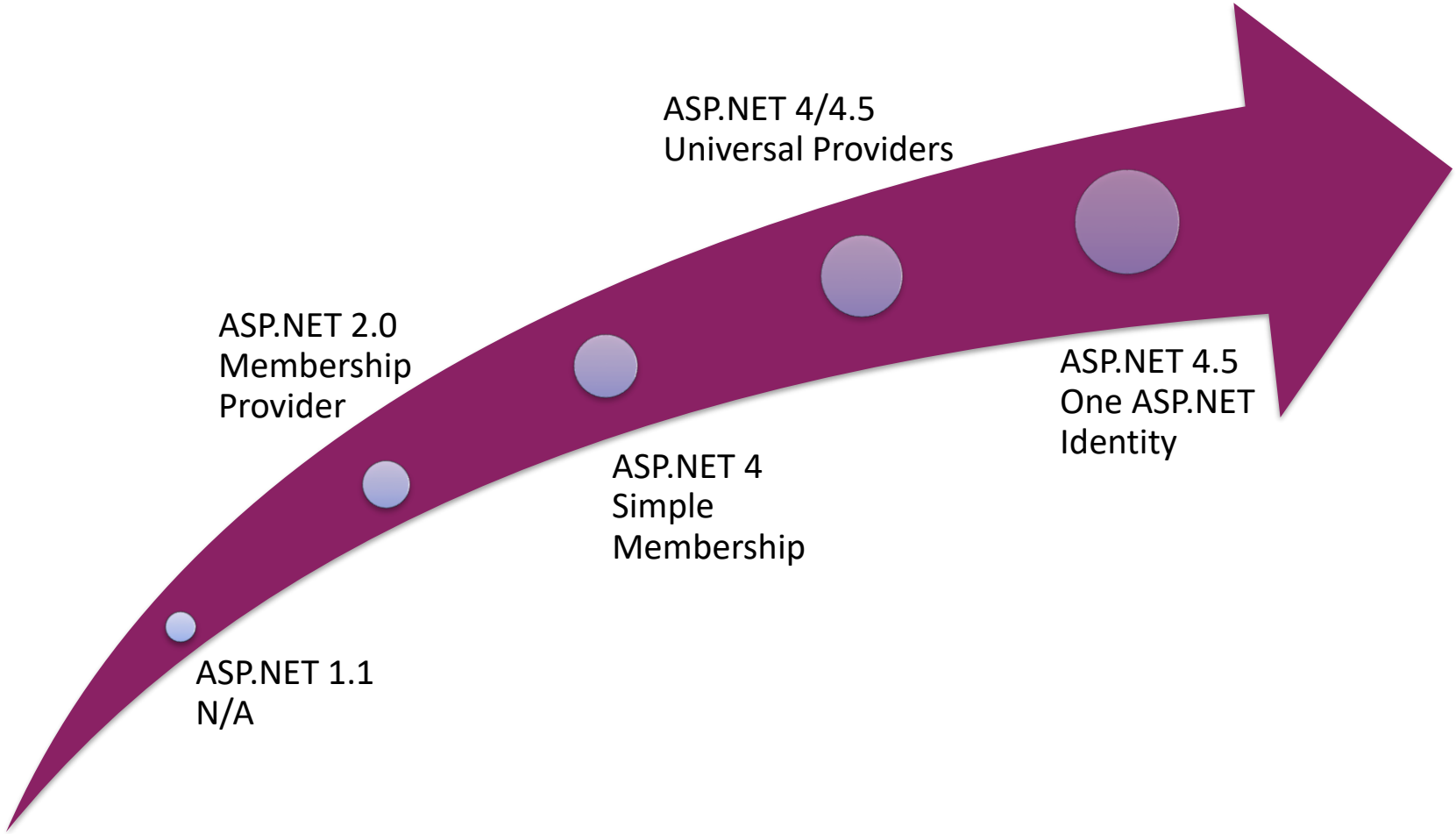
Authentifizierung / Authorisierung

Authentifizierung/Autorisierung

- Authentifikation klärt die **WER**-Frage
  - Ist der Benutzer jener Benutzer, der er vorgibt zu sein?
- Autorisierung klärt die **WAS**-Frage
  - Was darf der Benutzer tun?

- ASP.NET Core Identity ist die Basis-Komponente für Authentifizierung/Autorisierung mittels Middleware
  - User-Verwaltung (Registrierung, Login etc…)
  - Persistierung via Entity Framework
  - Umfangreiche Konfigurierungsmöglichkeiten
    (Regeln, Rollen, Claims, Passwörter, External Providers)

Historie


ASP.NET 1.1
N/A
ASP.NET 2.0
Membership
Provider
ASP.NET 4
Simple
Membership
ASP.NET 4/4.5
Universal Providers
ASP.NET 4.5
One ASP.NET
Identity

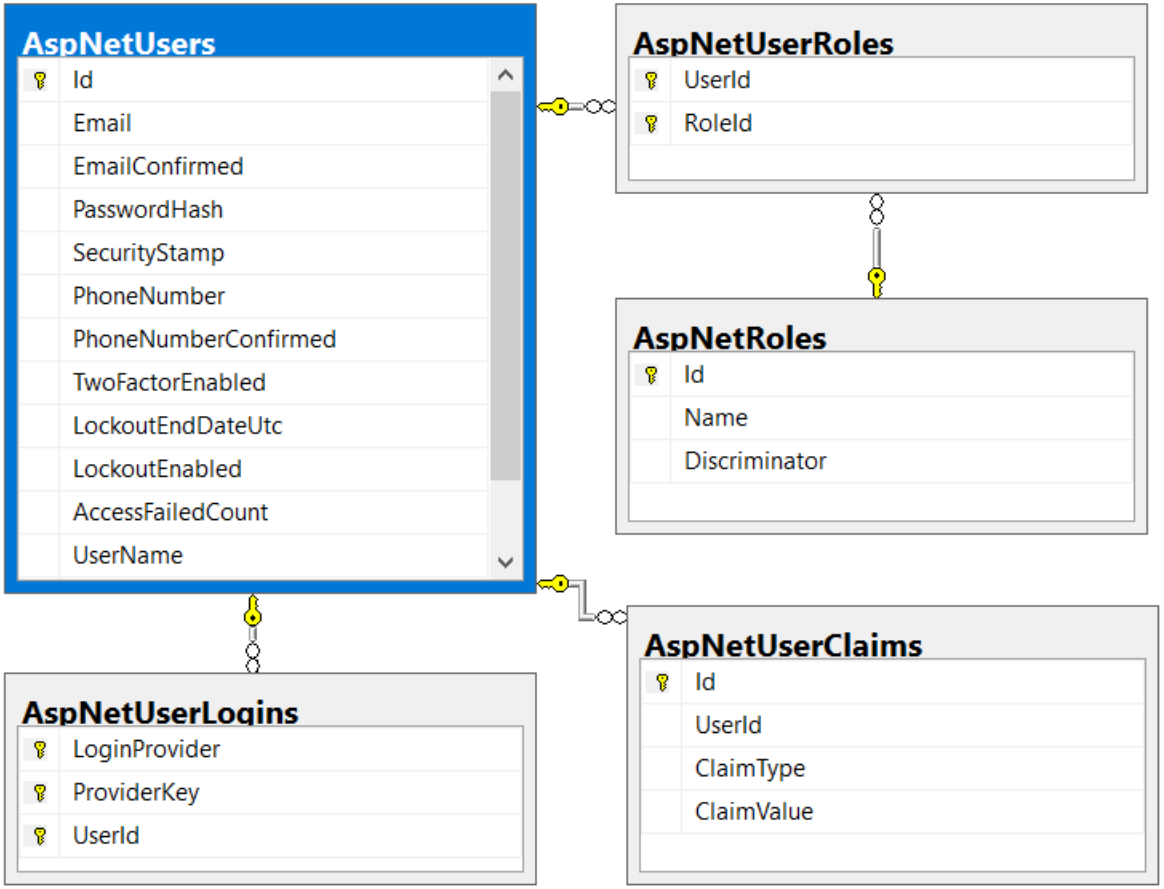
ASP.NET Identity

- One ASP.NET Identity System
  - MVC, Web Forms, WebAPI
- Role Provider
- Claims basierend
- Social Login Providers
- Windows Azure Active Directory
- OWIN Integration
- NuGet package

Neu in Identity 2.0

- Zwei-Factor-Authentifizerung
- Account Lockout
- Account Confirmation
- Password reset
- Erweiterter Password Validator

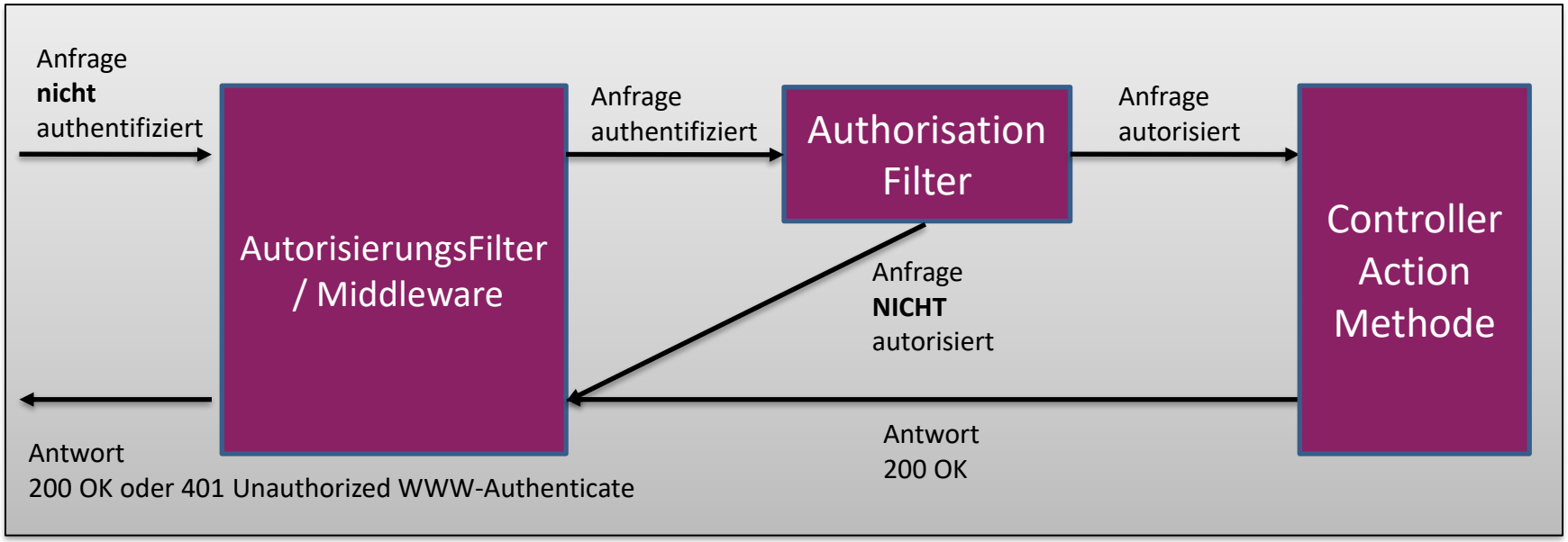
ASP.NET Identity Datenbank Schema



Authentifizierungsmöglichkeiten

Identity	Technologie	Szenario
Individual User Accounts	ASP.NET Identity, Social Providers	Internet
Active Directory / Windows Authentifizerung	IIS + Windows Authentifizierung	Intranet
Azure Active Directory	OpenID Connect	Cloud basierte Enterprise Apps
Keine Authentifizierung	-	Anonymer Zugriff

Authentifizierungs-/Autorisierungsablauf



Security Filters

- Authorize
  - Kontrolliert Wer den Controller/Action zugreifen darf via Eigenschaften Users / Roles / Claims

Hands On

Authentifizierung

JavaScript & JQuery

JavaScript & JQuery

- Bietet Interaktivität für die Webanwendung (Client/Server)
- Grundlage von AJAX
- MVC nutzt JS-Dateien in Views via
  - Inline JavaScript
  - JavaScript-Dateien
- Weitverbreiteste JS-Bibliothek
  - jQuery

JQuery

- Umfangreiche JavaScript-Bibliothek
- Funktionen zur DOM-Manipulation und – Navigation
- Visuelle Effekte
- Cross-Browser-Kompatibilität (jQuery 1.x)
- April 2013: jQuery 2.0 (aber kein IE 6-8 Support!)
- Juni 2016 JQuery 3.0

JQuery Zugriff auf HTML-Elemente

- Folgende CSS3 Selektoren stehen zur Auswahl von Elementen
  - Per Name => $("tr")
  - Per ID => $("#ElementID")
  - Per CSS-Klasse => $(".ueberschrift")

- Ereignisbehandlung

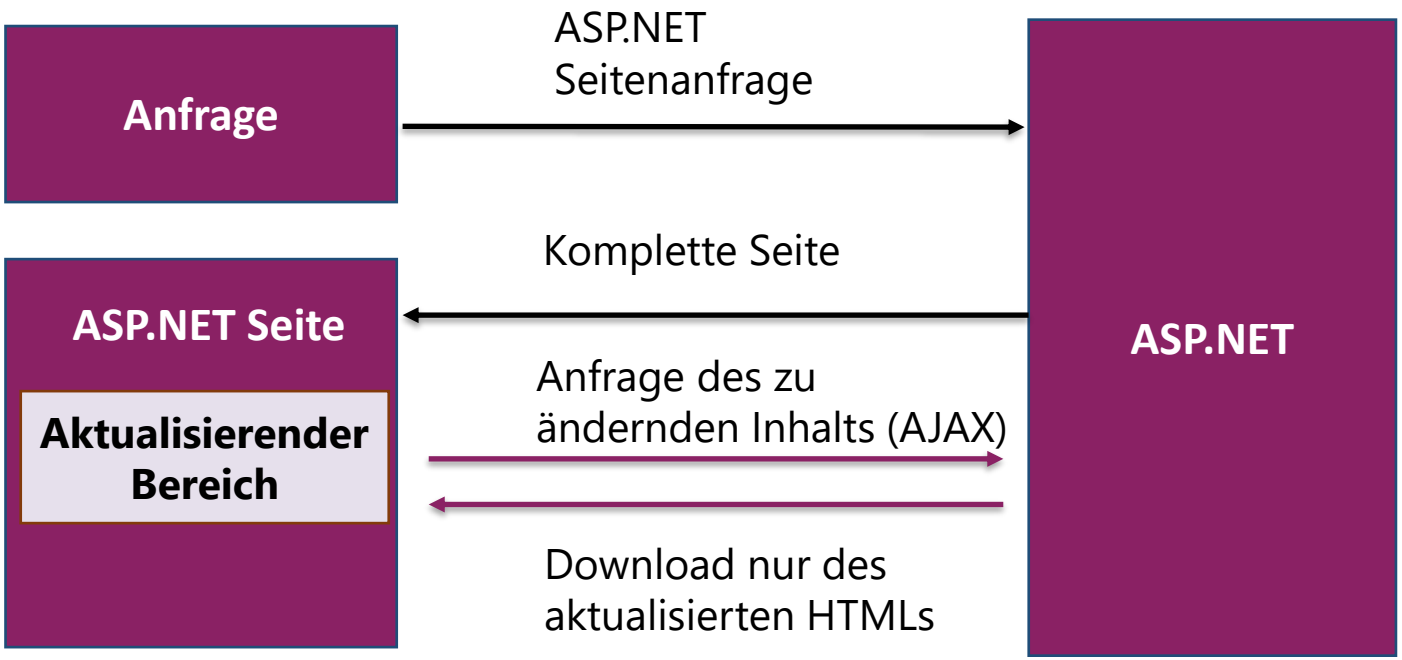
```
$(document).ready(function () {
        $("#HalloButton").click(function (event) {
         alert("Hallo Welt");
        });
}
```

AJAX-Anfragen mit JQuery

```
$.ajax({
        type:"GET",
        dataType: "json",
        url: "Home/GetAutorInfo",
        data: "{ ID: ‚123456‘}",
        contentType: "application/json; charset=utf-8",
        success: function(data) {
                        $("#autorName").html(data.autor.name);
        },
        failure: function(msg) {
                     alert(msg);
        }
});
```

- Alternativ Anfrage mit JSON-Rückgabe => **$.getJSON**(…)

AJAX

AJAX

- **A**synchronous **J**avaScript **a**nd **X**ML / Stichwort Web 2.0
- Erlaubt das Aktualisieren nur von gewissen Bereichen einer Webseite